

第三讲 并行计算体系结构 (2)

何克晶

kejinghe@gmail.com

华南理工大学

计算机科学与工程学院

目录

Flynn分类

内存访问模型

内存体系结构

处理器、协处理器与异构计算

更多系统案例

并行计算的分类

- 指令流/数据流分类法，即费林-Flynn分类法。
 - (1) 指令流 (Instruction Stream)：机器执行的指令序列。
 - (2) 数据流 (Data Stream)：指令调用的数据序列，包括输入数据和中间结果。
 - (3) 多倍性 (Multiplicity)：在系统性能瓶颈部件上同时处于同一执行阶段的指令或数据的最大可能个数

Flynn分类

- 两个独立的维度——指令流和数据流的不同组织方式，将计算机系统分为四类：
 - 单指令单数据流 (Single Instruction stream and Single Data stream , SISD) 系统
 - 单指令多数据流 (Single Instruction stream and Multiple-Data stream , SIMD) 系统
 - 多指令单数据流 (Multiple-Instruction stream and Single Data stream , MISD) 系统
 - 多指令多数据流 (Multiple-Instruction stream and Multiple-Data stream , MIMD) 系统

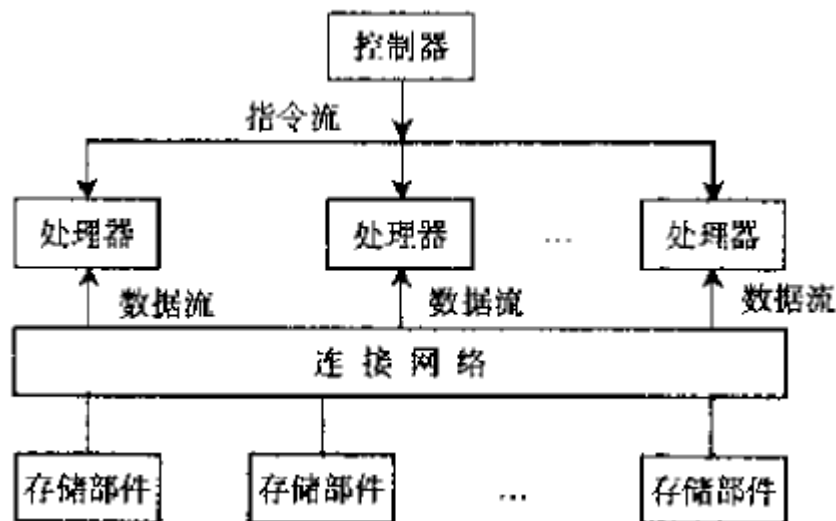
Flynn分类

- SISD: SISD系统是一种传统的顺序执行的单处理器计算机，它的硬件不支持任何形式的并行计算，所有的指令都是串行执行。在任何一个时钟周期内，CPU只能处理一个数据流，因此这种机器被称作单指令流单数据流机器
- SIMD: SIMD系统有多个处理单元，由单一的指令部件控制，按照同一指令流的要求为它们分配各不相同的数据流并进行处理。系统结构由一个控制器、多个处理器、多个存贮模块和一个互连总线（网络）组成
- MISD: MISD系统有多个处理单元，每个处理单元按照多条不同的指令要求同时对同一数据流及其处理输出的结果进行不同的处理，把一个单元的输出作为另一个单元的输入
- MIMD: MIMD系统又称为多处理机系统，是指能实现指令、数据作业、任务等各级全面并行计算的多机处理系统，可以将一个主任务分解为众多子任务并行执行以缩短工作时间

SIMD vs MIMD

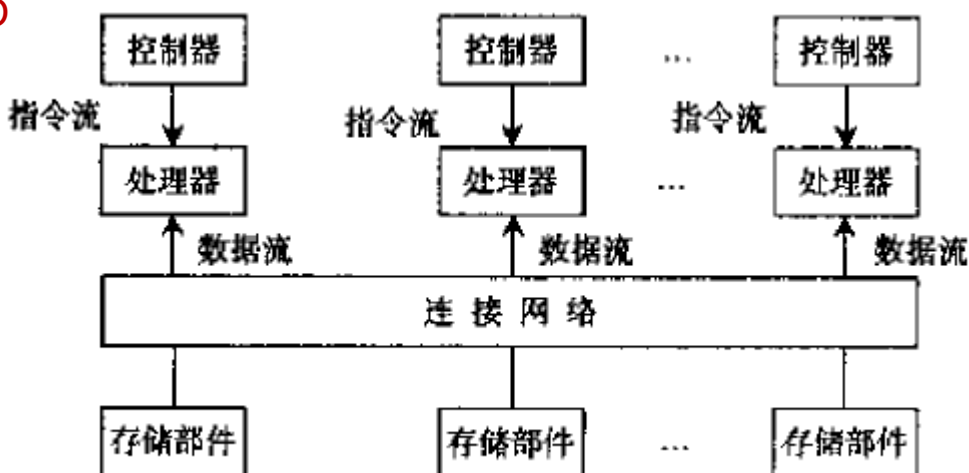
SIMD

计算机系统结构



MIMD

计算机系统结构



	SIMD	MIMD
控制器	一个	多个
处理器	多个	多个
存储部件	多个	多个
连接网络	有	有

• 区别1

- SIMD的各个处理器同步运行，即分别在来自存储器的不同数据流上并行执行相同的指令流，也就是说有一个指令流和多个数据流。
- MIMD的各个处理器异步运行，即在各自的数据流上执行自己的指令流，也就是说有多个指令流和多个数据流。

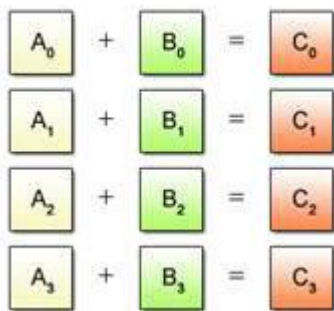
• 区别2

- SIMD的各个处理器同步使用连接网络，而MIMD的异步使用连接网络。

SIMD

- 可以通过SIMD，同时对多个数据元素执行单个操作
 - 人们有时会用向量化（vectorization）来表达对SIMD等向量化指令的使用

(a) Scalar Operation



```
function simple_sum(A)
    rst = zero(eltype(A))
    for x in A
        rst += x
    end
    return rst
end
```

```
21 A = rand(64, 64);
22
23 @btime simple_sum(A);
24
25 @btime simd_sum(A);
```

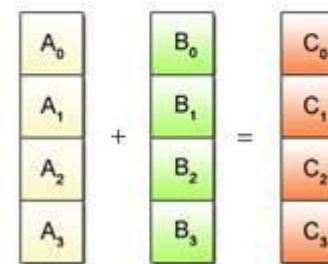
调试控制台

终端

```
julia> 正在运行 simd.jl
1.770 μs (1 allocation: 16 bytes)
130.125 ns (1 allocation: 16 bytes)
```

SIMD带来若干倍的性能提升

(b) SIMD Operation



```
function simd_sum(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += x
    end
    return rst
end
```

@simd宏用于告诉Julia：请尽量将这段代码编译为SIMD的版本

SIMD 的使用：加速比估计

- SIMD 的加速比取决于数据类型的位宽和 CPU 的架构
 - 大部分 CPU 支持 256 位 SIMD，即一次向量化运算的总位宽

4 x Float64 = 256位宽

```
25 A = rand(Float64, 64, 64);
26
27 @btime simple_sum(A);
28
29 @btime simd_sum(A);
```

调试控制台

终端

```
julia> 正在运行 simd.jl
1.770 μs (1 allocation: 16 bytes)
130.023 ns (1 allocation: 16 bytes)
```

8 x Float32 = 256位宽

```
31 A = rand(Float32, 64, 64);
32
33 @btime simple_sum(A);
34
35 @btime simd_sum(A);
```

调试控制台

终端

```
julia> 正在运行 simd.jl
1.770 μs (1 allocation: 16 bytes)
75.592 ns (1 allocation: 16 bytes)
```

Tip: 几乎所有的CPU均不支持16位浮点运算

SIMD 的使用: SIMD是有限制的

- 即使加了 @simd 宏, 也不一定会编译出带 SIMD 的版本

SIMD 代码中不允许条件分支, 即: if 判断或?: 三元运算符均不允许出现

```
function simple_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积

```
function simd_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @simd for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积: 带 @simd 宏

```
function simd_inbounds_dot(A, B)
    @assert axes(A) == axes(B)
    rst = zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        rst += A[i] * B[i]
    end
    return rst
end
```

向量内积: 带 @simd 与 @inbounds 宏

```
54 A, B=rand(4096), rand(4096);
55 @btime simple_dot(A,B);
56 @btime simd_dot(A,B);
```

调试控制台

终端

```
julia> 正在运行 simd.jl
1.790 μs (1 allocation: 16 bytes)
360.000 ns (1 allocation: 16 bytes)
```

Julia 矩阵取值时默认会进行下标越界检查, 该检查本质上是一个 if 判断, 阻碍了 SIMD 生效。@inbounds 宏可以用于取消该检查从而让 SIMD 生效

SIMD 的使用: SIMD是有限制的

- 即使加了 @simd 宏, 也不一定会编译出带 SIMD 的版本
 - SIMD 代码中不允许条件分支, 即: **if** 判断或 **XXX?AAA:BBB** 三元运算符均不允许出现

条件求和

```
function simple_dot_if(A,B)
    @assert axes(A)== axes(B)
    rst =zero(eltype(A))
    @inbounds for i in eachindex(A)
        a, b = A[i], B[i]
        if a*b > 0.5
            rst += a + b
        end
    end
    return rst
end
```

条件求和: 带@simd宏

```
function simd_dot_if(A,B)
    @assert axes(A)== axes(B)
    rst =zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        a, b = A[i], B[i]
        if a*b > 0.5
            rst += a + b
        end
    end
    return rst
end
```

```
function simd_dot_ifelse(A,B)
    @assert axes(A)== axes(B)
    rst =zero(eltype(A))
    @inbounds @simd for i in eachindex(A)
        a, b = A[i], B[i]
        rst += ifelse(a*b>0.5, a+b, zero(rst))
    end
    return rst
end
```

ifelse是一个**无分支形式**的条件判断函数。它与正常的if-else分支的差别在于: ifelse两种情况的值都会计算, 而if-else会根据条件选择性地计算一种情况下的值。

这本身代表一种优化思路: **引入额外计算来消除分支, 从而带来潜在的性能优化**

```
90 A, B=rand(4096), rand(4096);
91 @btime simple_dot_if(A,B);
92 @btime simd_dot_if(A,B);
93 @btime simd_dot_ifelse(A,B);
```

调试控制台

终端

```
julia> 正在运行 simd.jl
2.122 μs (1 allocation: 16 bytes)
558.602 ns (1 allocation: 16 bytes)
426.131 ns (1 allocation: 16 bytes)
```

SIMD 的使用：SIMD是有限制的

- 即使加了 @simd 宏，也不一定会编译出带 SIMD 的版本
 - 只有非常有限的基本指令支持SIMD，如：四则运算、位运算

带模求和

```
function simple_sum_mod(A)
    rst = zero(eltype(A))
    for x in A
        rst += x % 3
    end
    return rst
end
```

带模求和：带@simd宏

```
function simd_sum_mod(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += x % 3
    end
    return rst
end
```

基于查找表的带模求和

```
function mod3_lookup(n)
    lookup = Vector{Int}(undef, n)
    for i in 1:n
        lookup[i] = i % 3
    end
    return lookup
end

const LOOKUP_MOD3 = mod3_lookup(255)
function simd_sum_mod_lookup(A)
    rst = zero(eltype(A))
    @simd for x in A
        rst += @inbounds LOOKUP_MOD3[x]
    end
    return rst
end
```

```
126 A = rand(1:255, 64, 64);
127 @btime simple_sum_mod(A);
128 @btime simd_sum_mod(A);
129 @btime simd_sum_mod_lookup(A);
```

调试控制台 终端

```
julia> 正在运行 simd.jl
1.810 μs (1 allocation: 16 bytes)
1.810 μs (1 allocation: 16 bytes)
857.576 ns (1 allocation: 16 bytes)
```

优化思路：通过引入一些合理的限制和假设，不支持SIMD的运算可以转换为SIMD友好的运算

SIMD代码举例

- 原理知识：
 - SIMD (Single Instruction Multi Data) 单指令多数据流，支持将数据打包在一个大型寄存器中进行处理的一组指令集，在性能上相比标量操作吞吐量更大，适合数据密集型的操作；
 - Kunpeng 920支持ARMv8.2A指令集，有32个128bit的向量寄存器，流水线中有2个FSU单元可进行FP/ASIMD操作。
- 使用方法：
 - 编译器自动向量化优化，加入-ftree-vectorize，或编译优化设置为-O3，可添加-ftree-vectorizer-verbose参数查看编译器优化；
 - 使用Neon Intrinsics改写循环体，优化处理过程；
 - 引入OpenMP，使用OpenMP的关键词#pragma omp simd协助SIMD优化。
- 注意：
 - Kunpeng 920上使用SIMD优先推荐对整形/单精度浮点数据进行改写，长整形数据和双精度浮点类型效果相对不明显；
 - 针对短循环或者简单循环不建议进行SIMD优化，SIMD向量寄存器的LD/ST打包解包过程也会带来一定开销，短循环可能还会带来性能下降。

SIMD代码举例

```
#include <stdlib.h>
#include <math.h>
#include <string.h>

void vadd(int length, float* src)
{
    for (int i = 0; i < length; i++) {
        src[i] += src[i] * src[i];
    }
}

int main(int argc, char **argv)
{
    float *src = (float *)malloc(100000000 * sizeof(float));
    for (int i = 0; i < 100000000; i++) {
        src[i] = rand() * 100;
    }
    vadd(100000000, src);
}

Performance counter stats for './vadd':
    2,299.48 msec task-clock:u    # 0.99
    0 context-switches:u        # 0.00
    0 cpu-migrations:u          # 0.00
    389 page-faults:u           # 0.16
    5,761,850,685 cycles:u       # 2.50
    7,690,540,540 instructions:u # 1.33
    <not supported> branches:u
    3,234,224 branch-misses:u

    2.301245250 seconds time elapsed
    2.244850000 seconds user
    0.047914000 seconds sys

#include <stdlib.h>
#include <math.h>
#include <string.h>
#include <arm_neon.h>

static inline void vadd(int length, float* src)
{
    float32x4_t vsrc;
    float32x4_t vtemp;
    for (int i = 0; i < length / 4; i++) {
        vsrc = vld1q_f32(src + i * 4);
        vtemp = vmlaq_f32(vsrc, vsrc, vsrc);
        vst1q_f32(src + i * 4, vtemp);
    }
}

int main(int argc, char **argv)
{
    float *src = (float *)malloc(100000000 * sizeof(float));
    for (int i = 0; i < 100000000; i++) {
        src[i] = rand() * 100;
    }
    vadd(100000000, src);
}

Performance counter stats for './vadd_inline':
    2,130.46 msec task-clock:u    # 1.000 CPUs utilized
    0 context-switches:u        # 0.000 K/sec
    0 cpu-migrations:u          # 0.000 K/sec
    510 page-faults:u           # 0.239 K/sec
    5,366,448,557 cycles:u       # 2.519 GHz
    7,690,540,553 instructions:u # 1.43 insn per cycle
    <not supported> branches:u
    3,232,127 branch-misses:u

    2.131350040 seconds time elapsed
    2.048101000 seconds user
    0.080029000 seconds sys
```



SIMD小结

- SIMD 是CPU指令集并行：它可以带来近乎免费的性能加速，但使用条件比较严格
- 最佳实践：
- 一般 SIMD 仅发生在底层基础代码：高层代码几乎不可能满足 SIMD 的要求
- Julia
 - @simd 宏只在确实有效的时候才添加
 - @inbounds 宏只在确定下标不会越界，并且能够带来可观性能优势的时候才添加
 - @simd 只可能出现在嵌套 for 循环的最内层循环
 - 通过性能对比(@btime)来验证 @simd 的加速效果
 - 通过 @code_llvm 来检查生成的代码中 SIMD 是否生效
 - 升级 Julia 版本：编译器会做非常多好的 SIMD 优化
 - SIMD.jl允许手动的 SIMD 代码编写：对于特定底层算法实现可以带来更激进的优化策略
 - LoopVectorization.jl 提供 @turbo 宏：包含 AVX 以及 for 循环优化等综合性手段

目录

Flynn分类

内存访问模型

内存体系结构

处理器、协处理器与异构计算

更多系统案例

并行计算机体系结构

- 组成要素

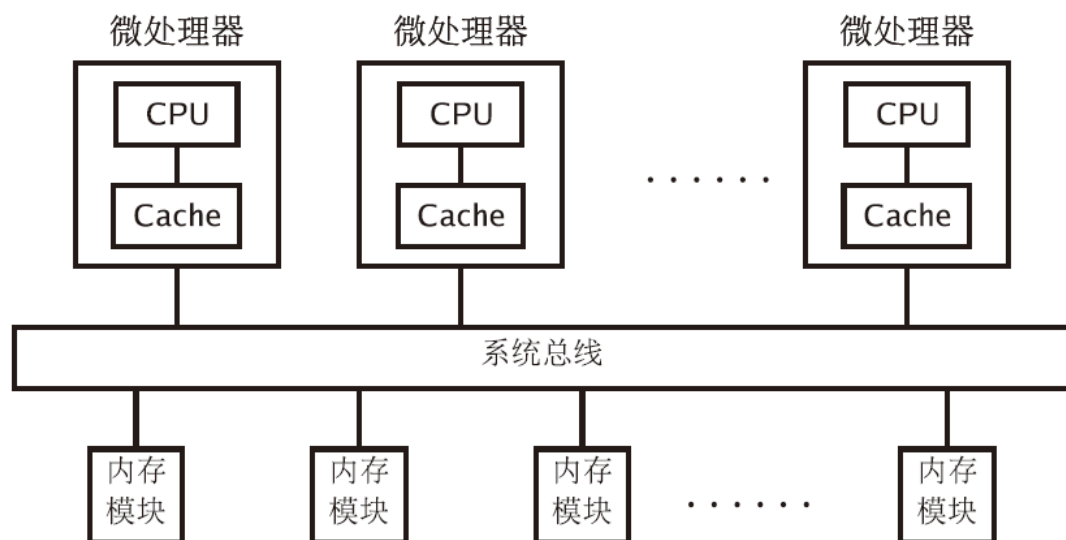
- 处理器 (processor) : 计算单元
- 互联网络 (interconnect network) : 连接
- 内存 (memory) : 多个存储模块组成

- 并行机的基本特征是具备多个计算单元和存储模块，各个模块通过互联网络耦合

- 根据耦合的紧密程度可分为紧耦合和松耦合。
- 不同的并行计算机，其各模块耦合的松紧程度可以有区别

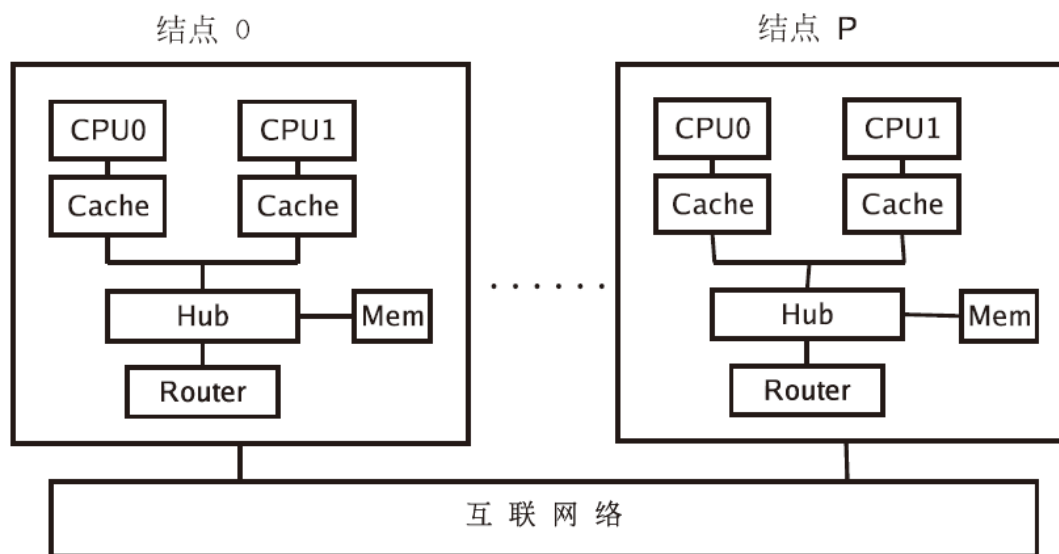
共享内存 vs 分布式内存

- 共享内存的多处理机
(multiprocessors) 体系



欠可靠、可扩展性较差

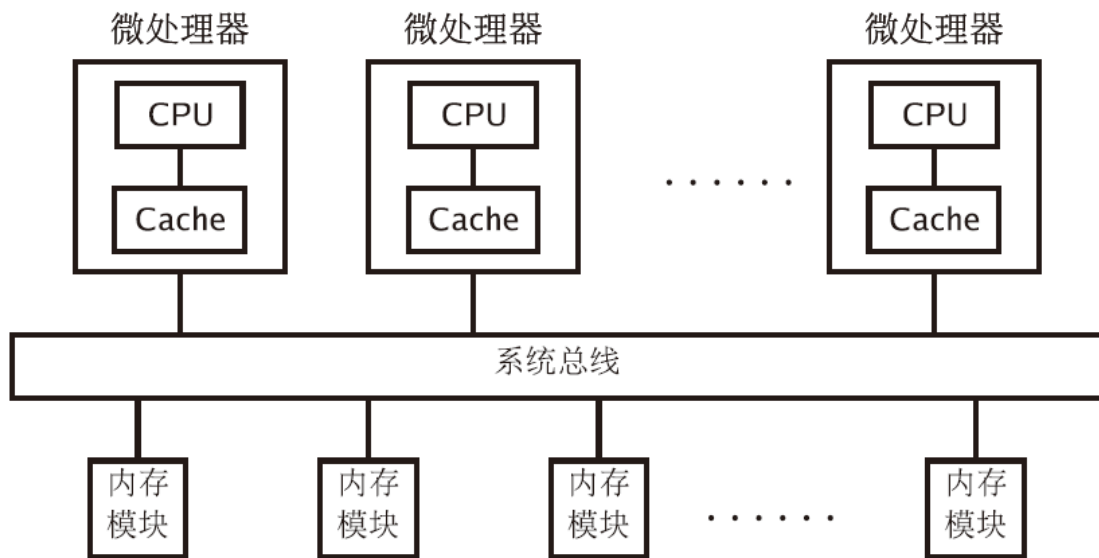
- 分布式内存的多计算机
(multicomputers) 体系



并行计算机访存模型

- UMA (Uniform Memory Access) 模型：均匀存储访问模型。
- NUMA (Non-Uniform Memory Access) 模型：非均匀存储访问模型。
- COMA(Cache-Only Memory Access)模型：全高速缓存存储访问。
- CC-NUMA (Coherent-Cache Nonuniform Memory Access) 模型：高速缓存一致性非均匀存储访问模型。
- NORMA (No-Remote Memory Access) 模型：非远程存储访问模型。

共享内存

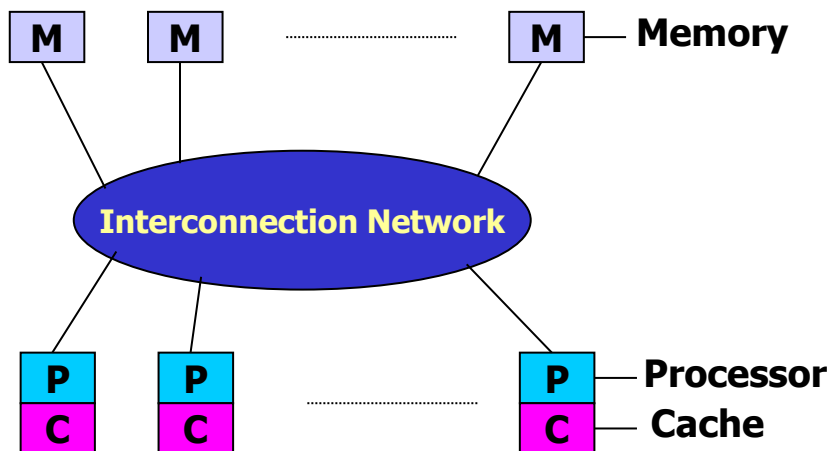


- 欠可靠、可扩展性较差。

- 通常也称为紧密耦合多处理机，它具有一个所有处理器都可以访问的全局物理内存。共享内存系统的特性有：
- (1) 对称性：系统中任何处理器都可以访问任何的内存单元和I/O设备
- (2) 单地址空间：内存中每一个位置在整个的内存地址范围内有一个唯一的地址
- (3) 低通信延迟：处理器间的通信可以利用共享内存来进行数据交换
- (4) 高速缓存及其一致性：多级高速缓存可以支持数据的局部性，而其一致性可由硬件来增强

UMA (Uniform Memory Access)

- 均匀存储访问 (UMA)



- 物理存储器被所有处理器均匀共享;
- 所有处理器访问任何存储字取相同的时间;
- 每台处理器可带私有高速缓存;
- 外围设备(I/O)也可以一定形式共享。
- 发生访存竞争时, 仲裁策略平等对待每个结点, 即每个结点机会均等;

- 对称多处理 (Symmetric Multiprocessing , SMP)

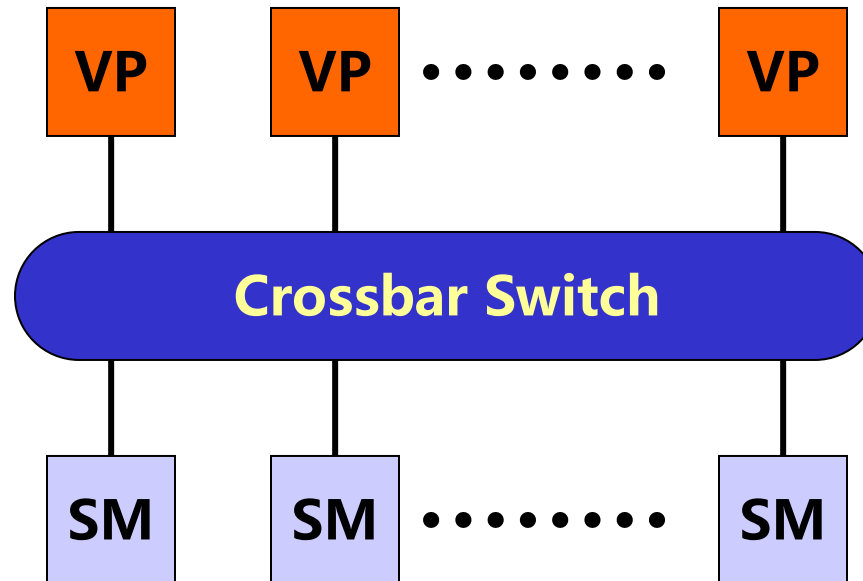
- SMP使用的是微处理器和高速缓存

- 并行向量处理机 (Parallel Vector Processoe, PVP)

- PVP使用的是高性能专门设计定制的向量处理器 (Vector Processor, VP)

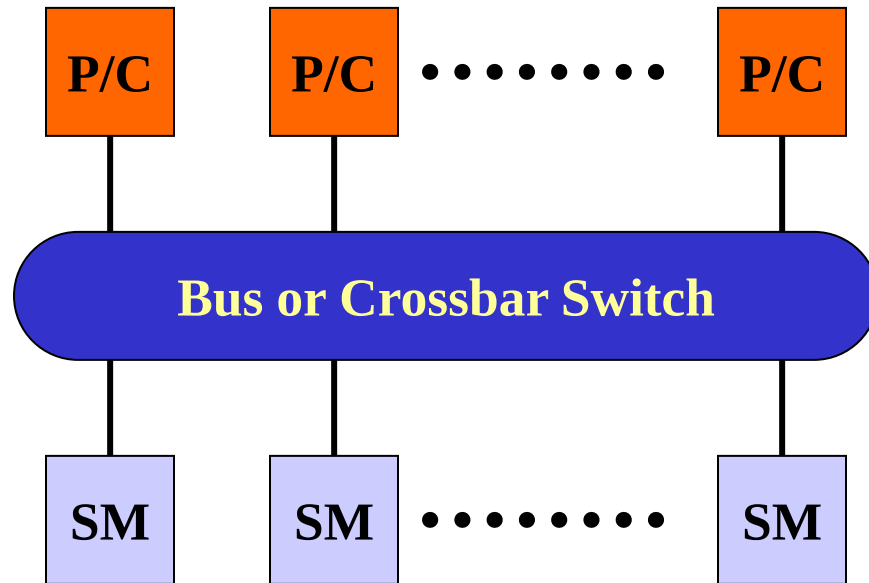
PVP (Parallel Vector Processor)

VP : Vector Processor SM : Shared Memory



SMP (Symmetric Multiprocessing)

- 内存模块和处理器对称地分布在互联网络的两侧;
- 内存访问属典型的均匀访问模型。



- ✓ 扩展性较差
- ✓ 可靠性较差
- ✓ 内存访问瓶颈

SMP的特点

- 对称共享存储

- 系统中任何处理器均可直接访问任何存储模块中的存储单元和I/O 模块，且访问的延迟、带宽和访问成功的概率是一致的。所有内存单元统一编址。各个处理器之间的地位等价，不存在任何特权处理器。操作系统可在任意处理器上运行。

- 单一的操作系统映像

- 全系统只有一个操作系统驻留在共享存储器中，它根据各个处理器的负载情况，动态地分配各个进程到各个处理器，并保持各处理器间的负载平衡。

- 局部高速缓存cache 及其数据一致性

- 每个处理器均配备局部cache，它们可以拥有独立的局部数据，但是这些数据必须与存储器中的数据保持一致。

SMP的特点 (2)

- 低通信延迟

- 各进程通过读/写操作系统提供的共享数据缓存区来完成处理器间的通信，其延迟通常小于网络通信的延迟。

- 共享总线带宽

- 所有处理器共享总线的带宽，完成对内存模块和I/O 模块的访问。

- 支持消息传递、共享存储并行程序设计。

SMP的缺点

- 欠可靠

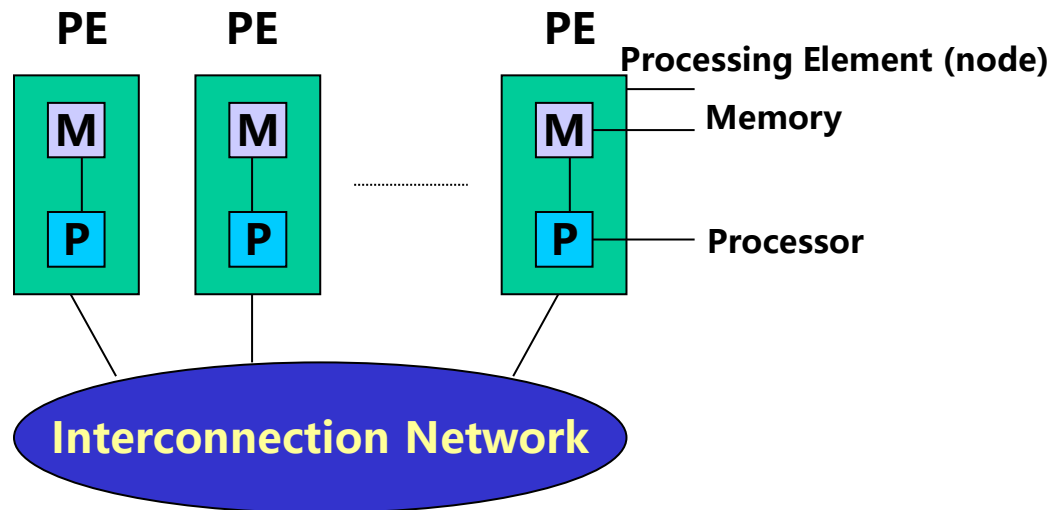
- 总线、存储器或操作系统失效可导致系统崩溃。

- 可扩展性 (scalability) 较差

- 由于所有处理器共享总线带宽，而总线带宽每3 年才增加2 倍，跟不上处理器速度和内存容量的增加步伐，因此，SMP 并行机的处理器个数一般少于32 个，且只能提供每秒数百亿次的浮点运算性能。

NUMA (Non-Uniform Memory Access)

- 非均匀存储访问 (NUMA)



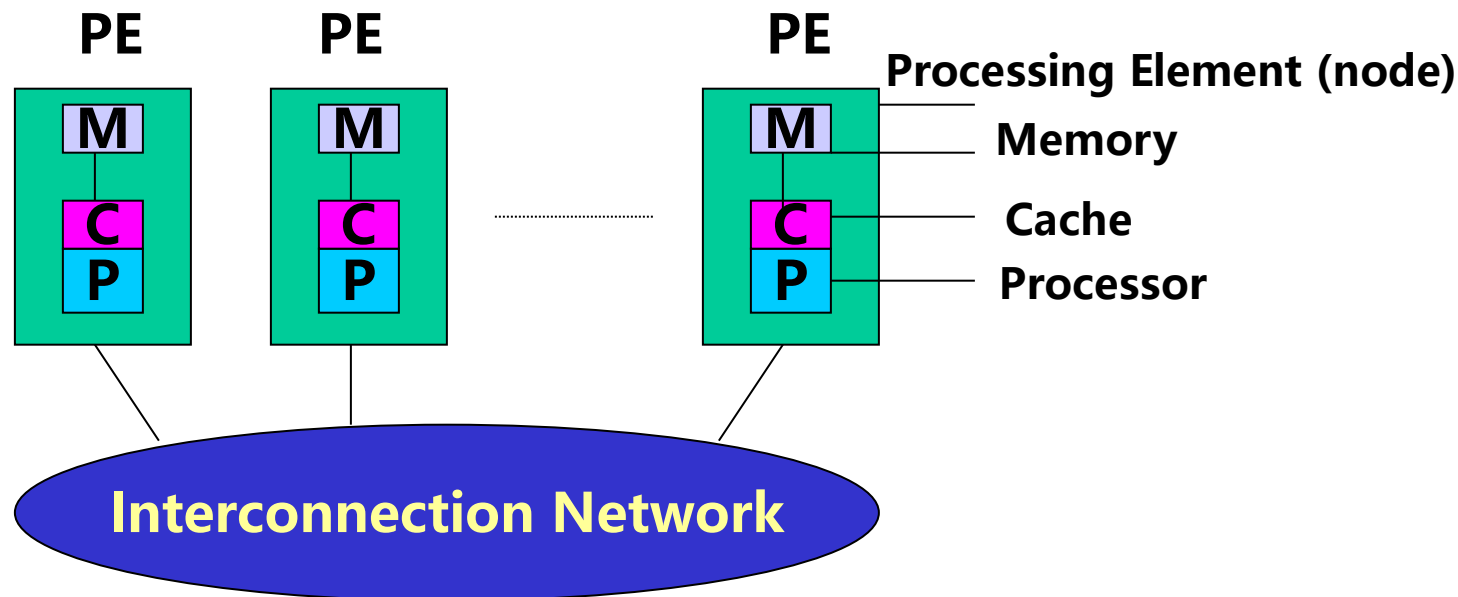
- 如果缓存一致性能得到维护，那么也可以称之为高速缓存一致性NUMA (Cache Coherent NUMA, CC-NUMA)，反之可称为高速缓存非一致性NUMA (Non- Cache Coherent NUMA, NCC-NUMA)

- 被共享的存储器在物理上是分布在所有的处理器中的，其所有本地存储器的集合就组成了全局地址空间；
- 处理器访问存储器时间是不一样的；
- 每台处理器照例可带私有高速缓存。

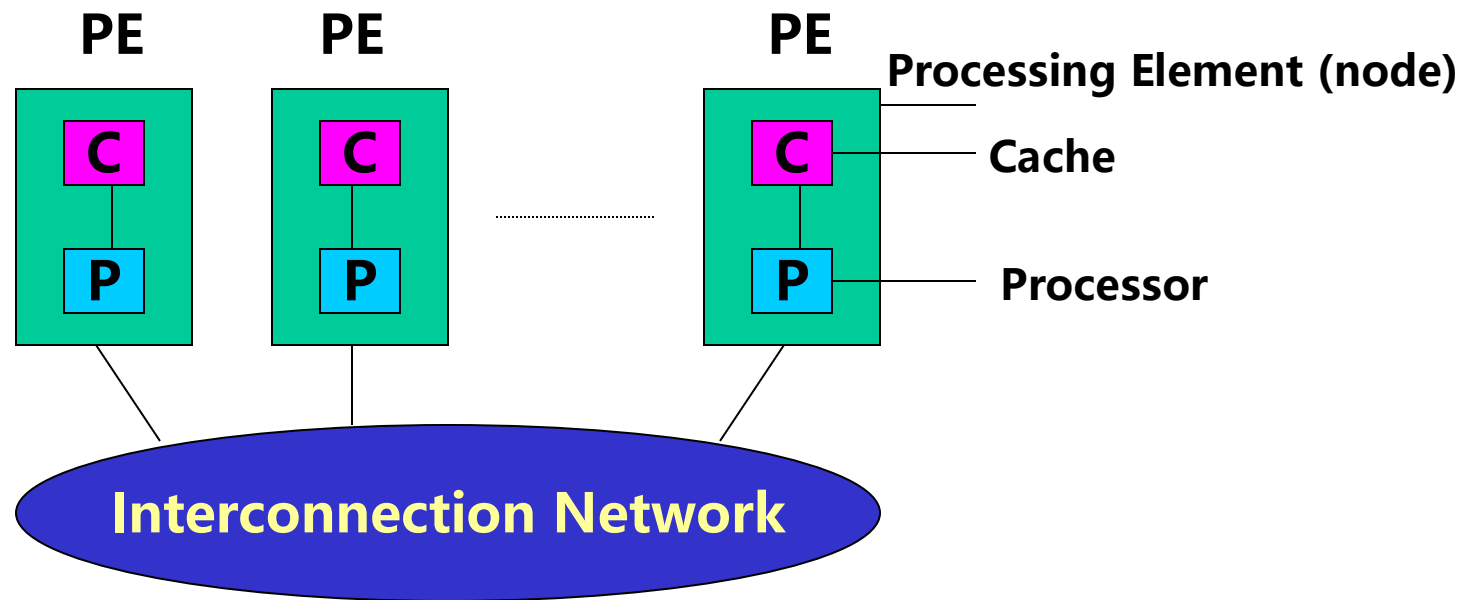
CC-NUMA (高速缓存一致性非均匀存储访问模型)

- 其特点是:

- 大多数使用基于目录的高速缓存一致性协议;
- 保留SMP结构易于编程的优点, 也改善常规SMP的可扩展性;
- CC-NUMA实际上是一个分布共享存储的DSM多处理机系统;
- 它最显著的优点是程序员无需明确地在节点上分配数据, 系统的硬件和软件开始时自动在各节点分配数据, 在运行期间, 高速缓存一致性硬件会自动地将数据迁移至要用到它的地方。



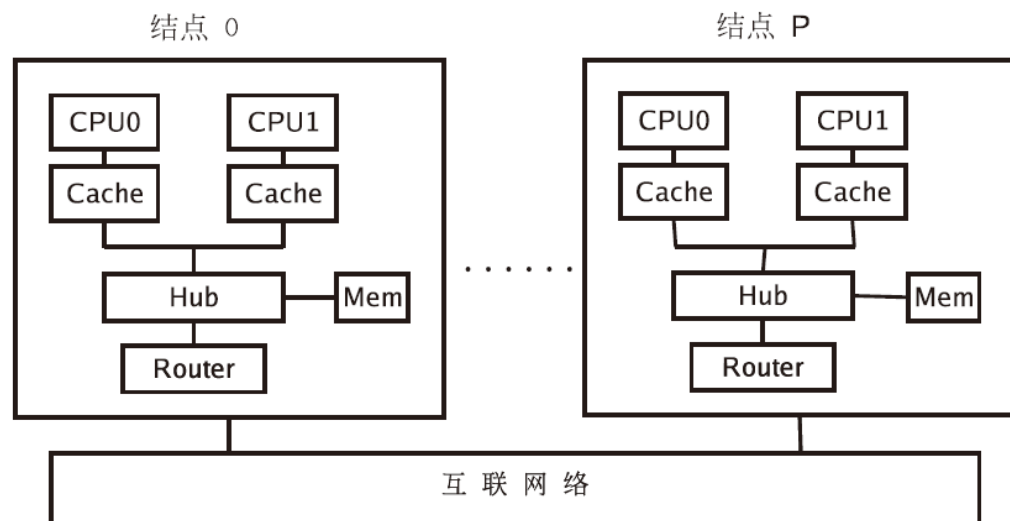
COMA (全高速缓存存储访问)



- 特例：全高速缓存存储访问（Cache-Only Memory Access, COMA）模型，COMA各个处理器节点没有存储层次结构，所有节点的高速缓存构成了全局地址空间
- Example: KSR - 1

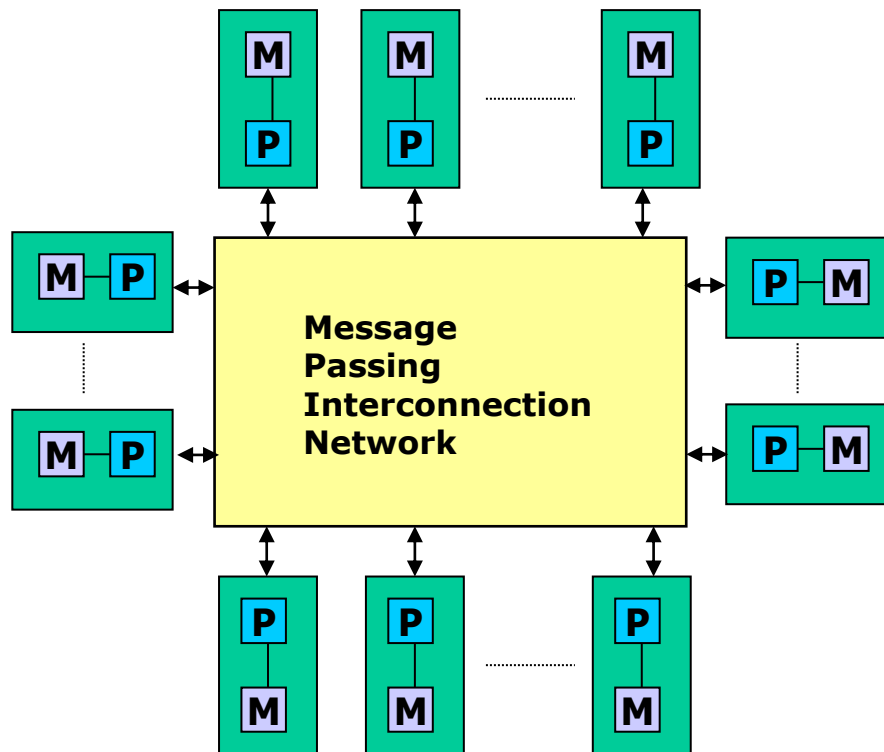
分布式内存

- 分布式内存的多计算机
(multicomputers) 体系



- 分布式内存系统中处理器都有各自的内部寄存器，一个核内的内存地址对其他核不可见，只能由该处理器所访问，对于所有CPU都没有单一全局地址空间的概念，这类的分布式计算机系统称为非远程存储访问 (No-Remote Memory Access, NORMA)

NORMA (非远程存储访问)



- 优点:

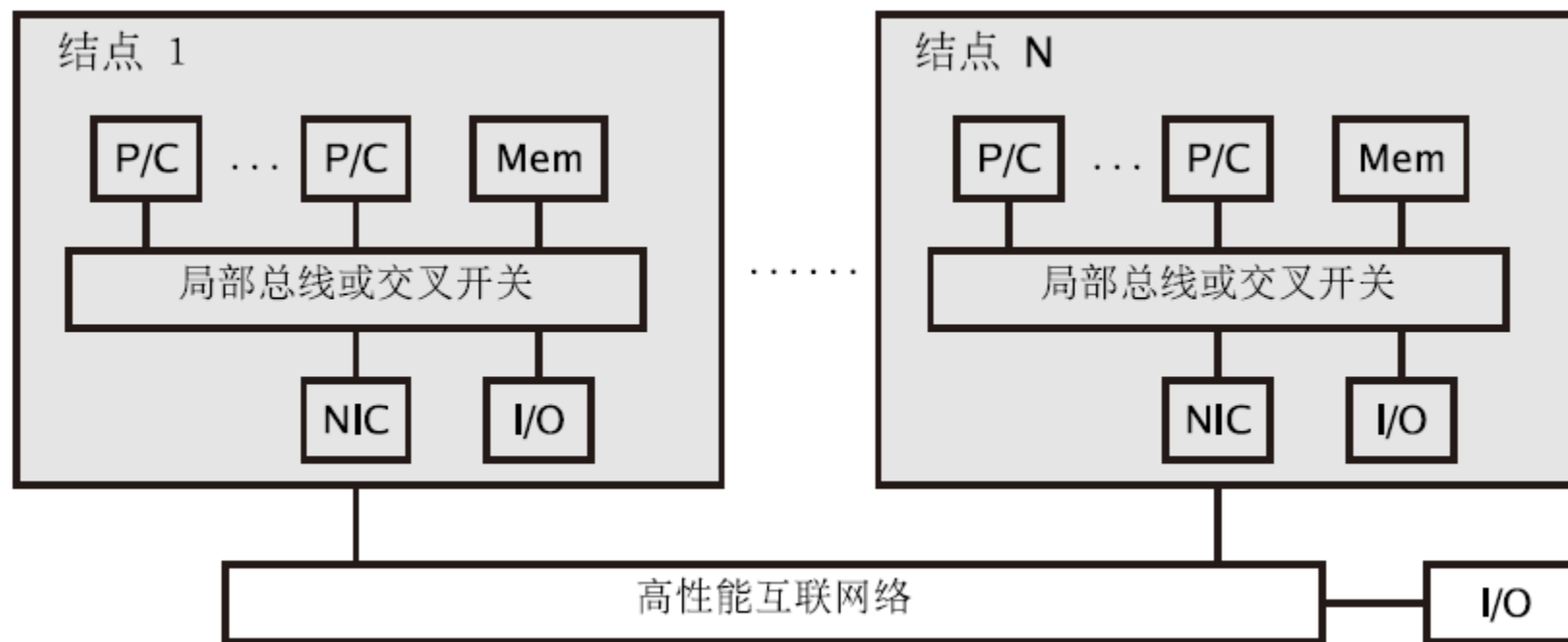
- (1) 内存可以随着CPU的数量进行扩展, 增加处理器数量将使内存的大小等比例增加。
- (2) 各个处理器可以无冲突地快速访问自己的内存, 也不存在维护缓存一致性的开销。
- (3) 成本效益上, 可以使用商用、现成的处理器和网络。

- 局限性:

- (1) 程序员将要负责所有处理器间数据通信相关的细节问题。
- (2) 很难从基于全局内存空间的数据结构上建立起分布式内存管理的映射。
- (3) 非一致的内存访问时间使得驻留在远程节点上的数据比节点本地数据的访问需要更长的时间。

大规模并行处理机 (Massively Parallel Processor , MPP)

- 由大规模“紧密”互连的节点组成
- 内存访问属非远程存储访问模型 (NORMA)
- 也被称为“message-passing” 系统

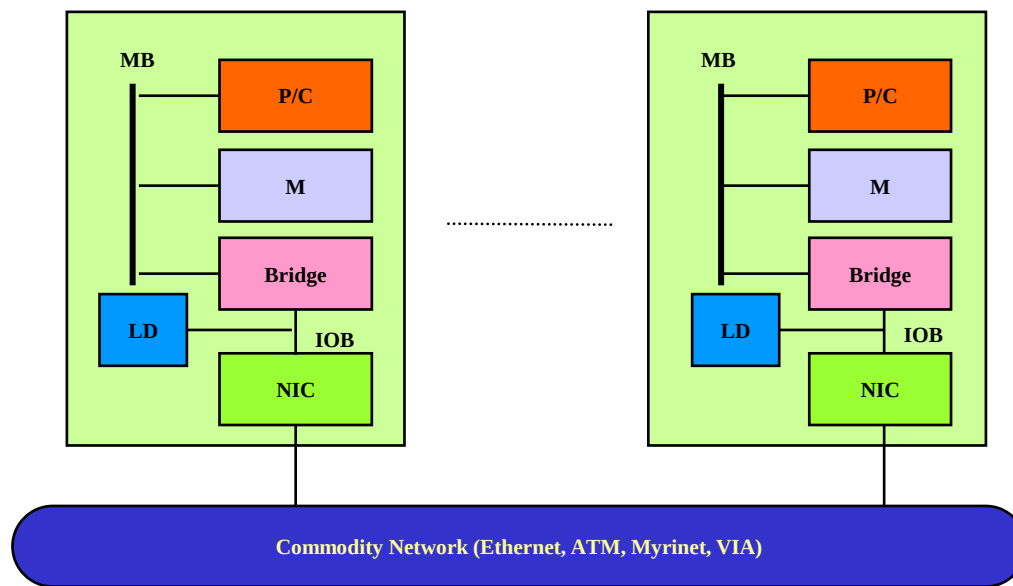


MPP的优点

- 由数百个乃至数千个计算结点和I/O 结点组成，每个结点相对独立，并拥有一个或多个微处理器。
 - 这些结点配备有局部cache，并通过局部总线或互联网络与局部内存模块和I/O 设备相连接。
 - 这些结点由局部高性能网卡(NIC) 通过高性能互联网络相互连接。
 - 各个结点均拥有不同的操作系统映像。
 - 一般情况下，用户可以将作业提交给作业管理系统，由它负责调度当前最空闲、最有效的计算结点来执行该作业。但是，MPP也允许用户登录到某个特定的结点，或在某些特定的结点上运行作业。
 - 各个结点间的内存模块相互独立，且不存在全局内存单元的统一硬件编址。
 - 仅支持消息传递或者高性能Fortran 并行程序设计，不支持全局共享的OpenMP 并行程序设计模式。

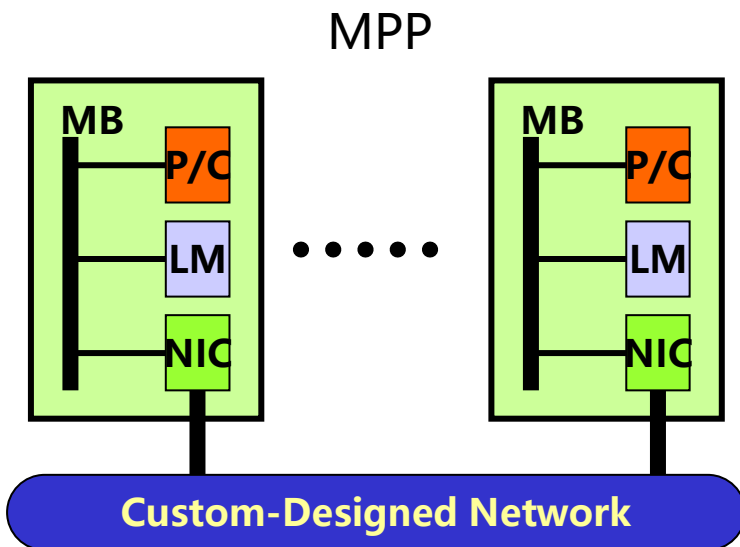
集群/机群 (Cluster) 、COW

- 松耦合。分布式存储，MIMD，工作站+商用互连网络，每个节点是一个完整的计算机，有自己的磁盘和操作系统
- 优点：
 - 投资风险小
 - 系统结构灵活
 - 性能/价格比高
 - 能充分利用分散的计算资源
 - 可扩充性好
- 问题
 - 通信性能
 - 并行编程环境
- 例子：Berkeley NOW, Alpha Farm, FXCOW

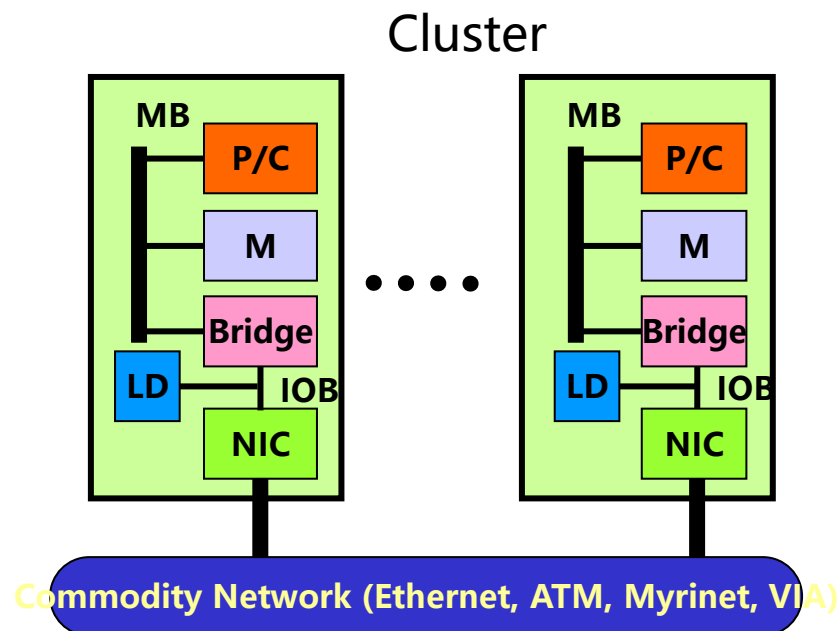


MPP vs Cluster

- Cluster的每个结点都是一台完整的计算机。Cluster的每个结点上都有完整的操作系统，而MPP的每个结点上通常只有操作系统的微核。Cluster的网络和操作系统均不是定制的。
- Cluster的每个结点内有本地磁盘，而MPP的结点内没有。
- Cluster各结点的网络接口是连接到I/O总线上的（松耦合），而MPP各结点的网络接口是连接到存储总线上的（紧耦合）。

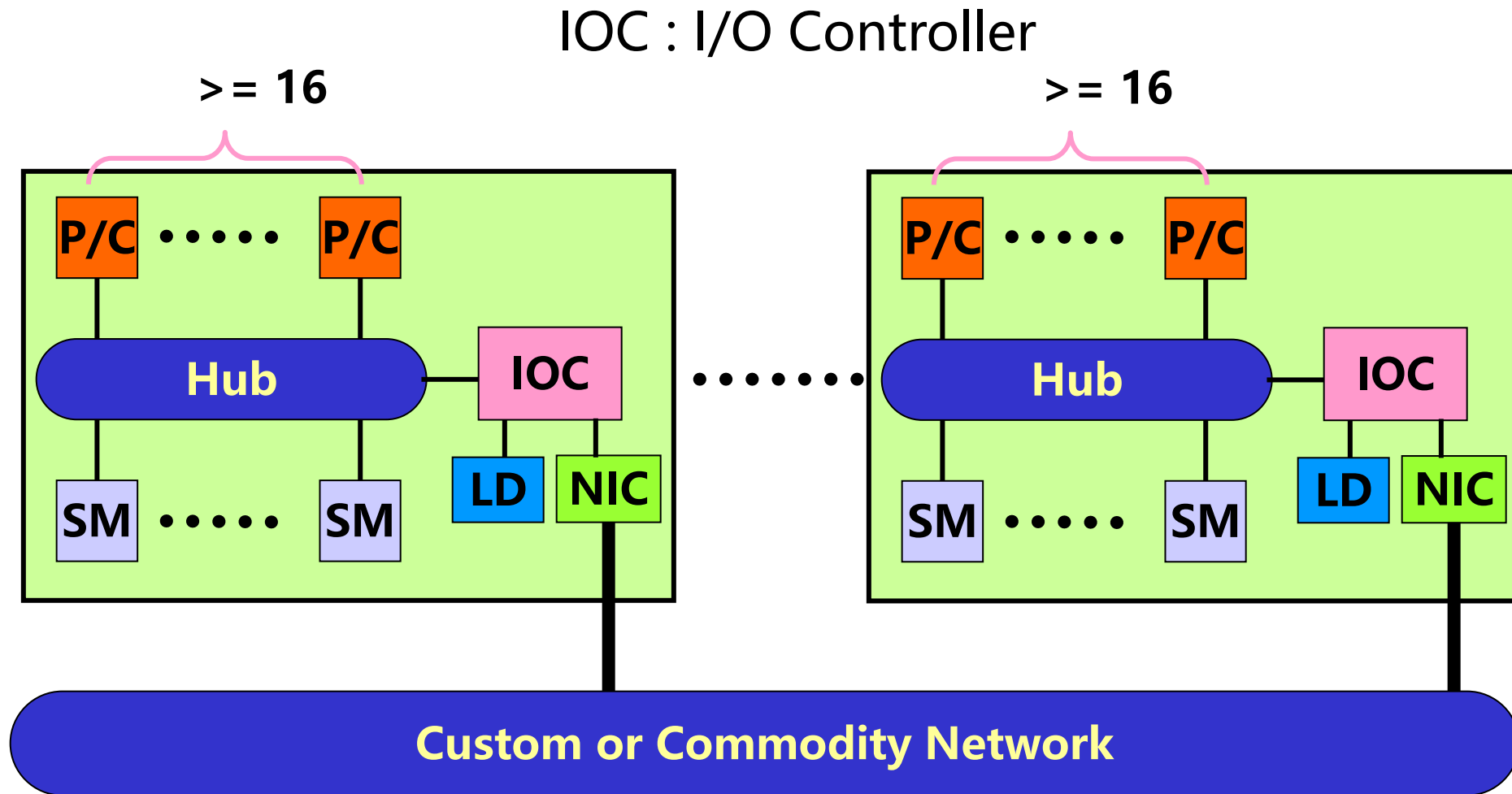


其中MB：存储器总线，P/C：处理器和高速缓存，NIC：网络接口电路，LM：表示本地存储器。



其中LD：本地磁盘，B：存储总线与I/O总线的接口，IOB：I/O总线。

Constellation (星群)



SMP\MPP\机群比较

系统特征	SMP	MPP	机群
节点数量(N)	$\leq O(10)$	$O(100)-O(1000)$	$\leq O(100)$
节点复杂度	中粒度或细粒度	细粒度或中粒度	中粒度或粗粒度
节点间通信	共享存储器	消息传递 或共享变量 (有DSM时)	消息传递
节点操作系统	1	N(微内核) 和1个主机OS(单一)	N (希望为同构)
支持单一系统映像	永远	部分	希望
地址空间	单一	多或单一 (有DSM时)	多个
作业调度	单一运行队列	主机上单一运行队列	协作多队列
网络协议	非标准	非标准	标准或非标准
可用性	通常较低	低到中	高可用或容错
性能/价格比	一般	一般	高
互连网络	总线/交叉开关	定制	商用

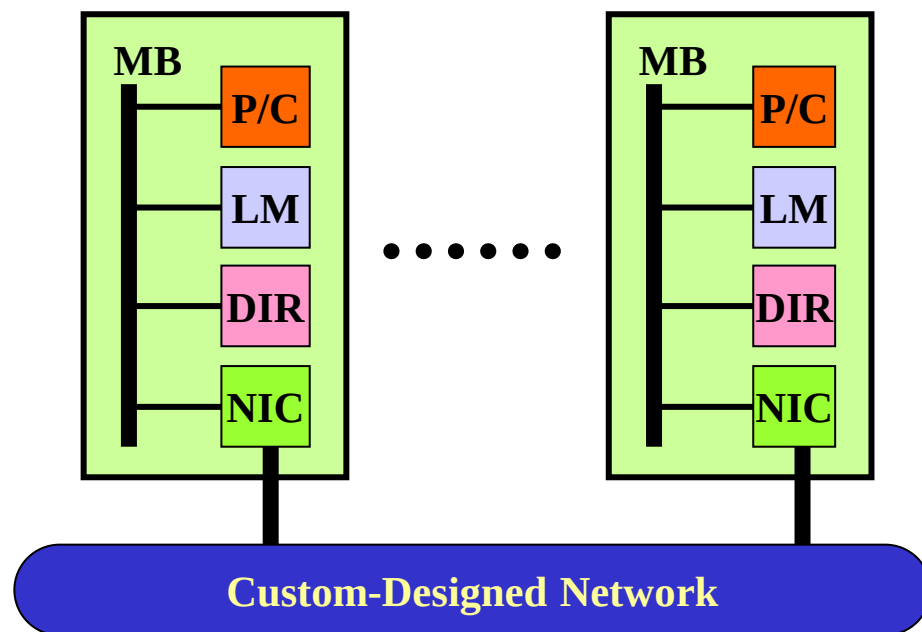
DSM (Distributed Shared Memory, DSM)

- 内存模块局部在各个结点内部，并被所有结点共享。这样，可以较好地改善对称多处理共享存储并行机的可扩展能力
- 内存访问属非一致内存访问模型（NUMA）。

MB : Memory Bus

NIC : Network Interface Circuitry

DIR : Cache Directory



DSM的特点（与SMP的相同点）

- 单一的操作系统映像。同SMP
- 基于cache 的数据一致性。同SMP
- 低通信延迟（同SMP） 与高通信带宽
 - 专用的高性能互联网络使得结点间的延迟很小
 - 通信带宽可以扩展。
- 支持消息传递、共享存储并行程序设计。同SMP

DSM的特点 (2)

- 并行机以结点为单位，每个结点包含一个或多个CPU，每个CPU 拥有自己的局部cache，并共享局部存储器和I/O设备，所有结点通过高性能互联网络相互连接
- 物理上分布存储
 - 内存模块分布在各结点中，并通过高性能互联网络相互连接，避免了SMP 访存总线的带宽瓶颈，增强了并行机的可扩展能力。
- 单一的内存地址空间
 - 尽管内存模块分布在各个结点，但是，所有这些内存模块都由硬件进行统一编址，并通过互联网络连接形成了并行机的共享存储器。各个结点既可以直接访问本地局部内存单元，又可以直接访问其他结点的局部内存单元。

DSM的特点 (3)

- 非一致内存访问 (NUMA) 模式

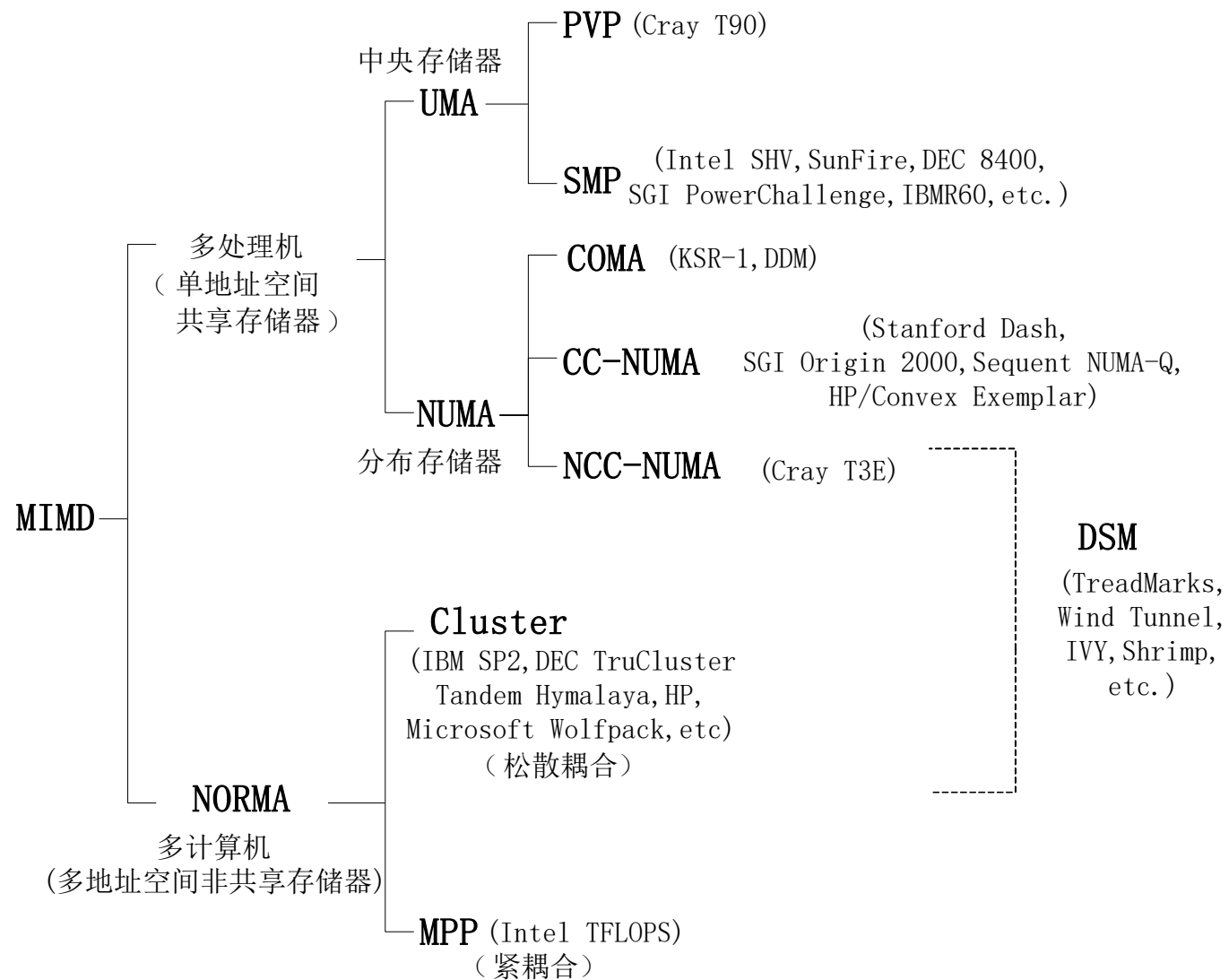
- 由于远端访问必须通过高性能互连网络，而本地访问只需直接访问局部内存模块，因此，远端访问的延迟一般是本地访问延迟的3 倍以上。

- 可扩展性强

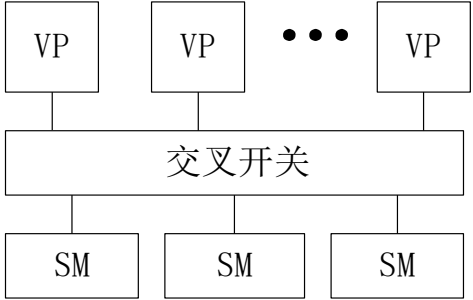
- 可扩展到数百个结点，能提供每秒数千亿次的浮点运算性能。

- 但是，由于受cache 一致性要求和互连网络性能的限制，当结点数目进一步增加时，DSM 并行机的性能也将下降。

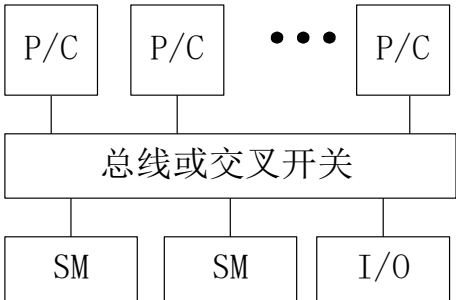
内存体系



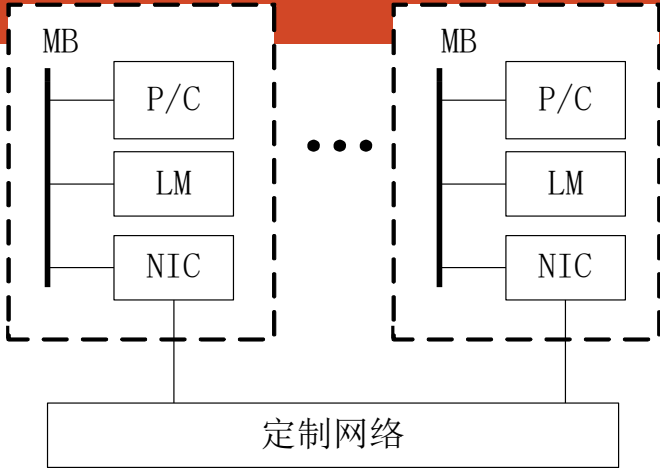
并行计算机结构模型



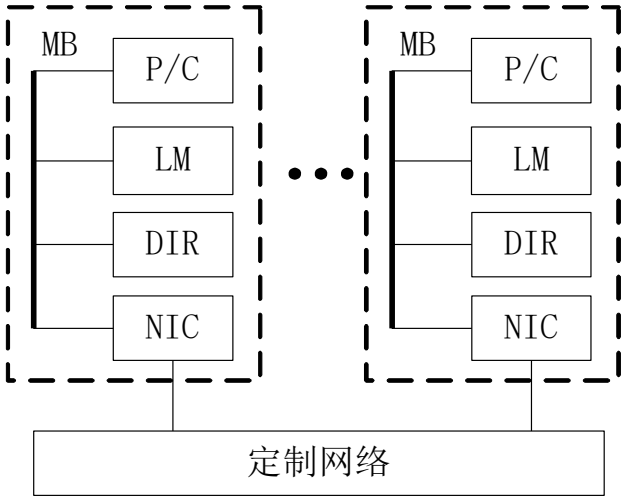
(a) PVP



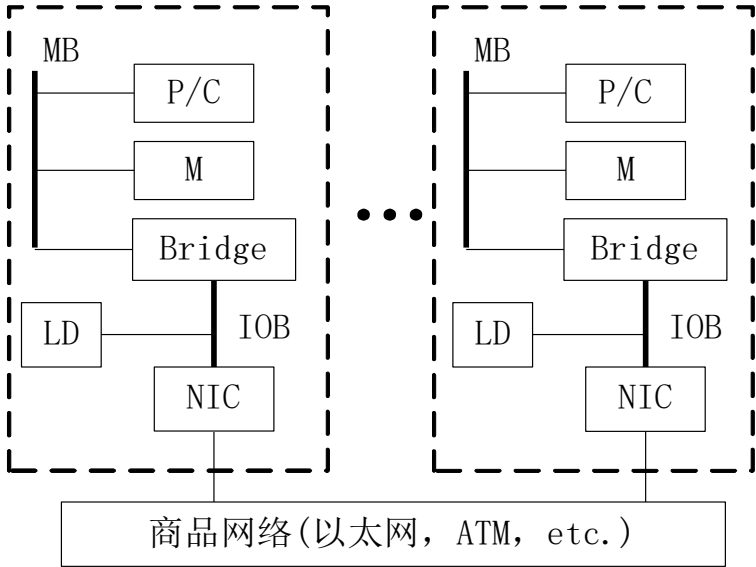
(b) SMP



(c) MPP



(d) DSM



(e) COW

五种结构特性一览表

属性	PVP	SMP	MPP	DSM	Cluster、COW
结构类型	MIMD	MIMD	MIMD	MIMD	MIMD
处理器类型	专用定制	商用	商用	商用	商用
互连网络	定制交叉开关	总线、交叉开关	定制网络	定制网络	商用网络 (以太 ATM)
通信机制	共享变量	共享变量	消息传递	共享变量	消息传递
地址空间	单地址空间	单地址空间	多地址空间	单地址空间	多地址空间
系统存储器	集中共享	集中共享	分布非共享	分布共享	分布非共享
访存模型	UMA	UMA	NORMA	NUMA	NORMA
代表机器	Cray C-90, Cray T-90, 银河1号	IBM R50, SGI Power Challenge, 曙光1号	Intel Paragon, IBMSP2, 曙光 1000/2000	Stanford DASH, Cray T 3D	Berkeley NOW, Alpha Farm

目录

Flynn分类

内存访问模型

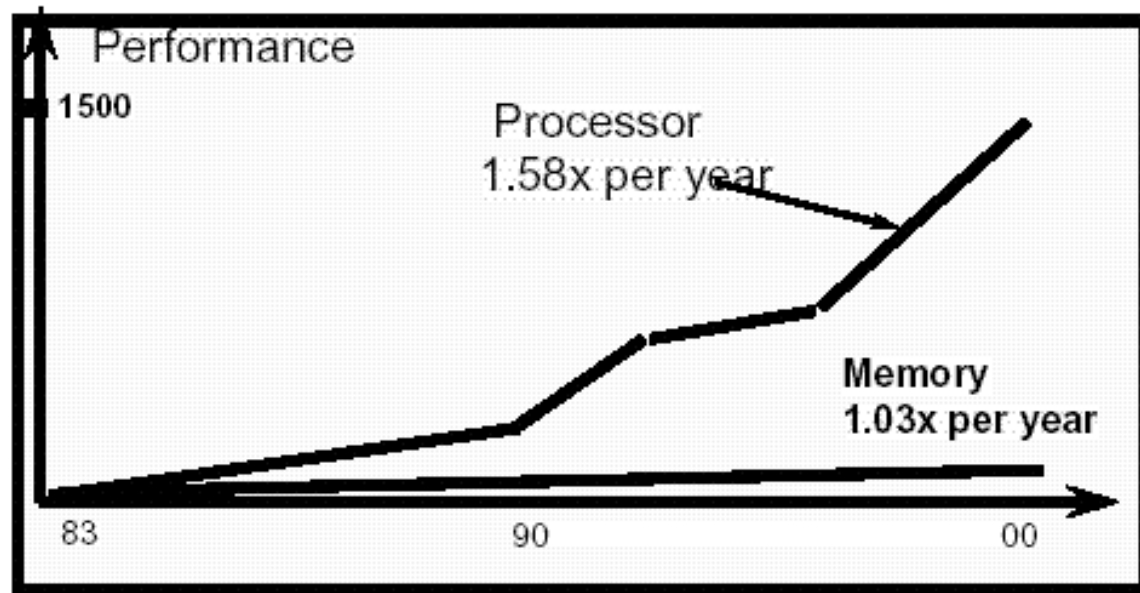
内存体系结构

处理器、协处理器与异构计算

更多系统案例

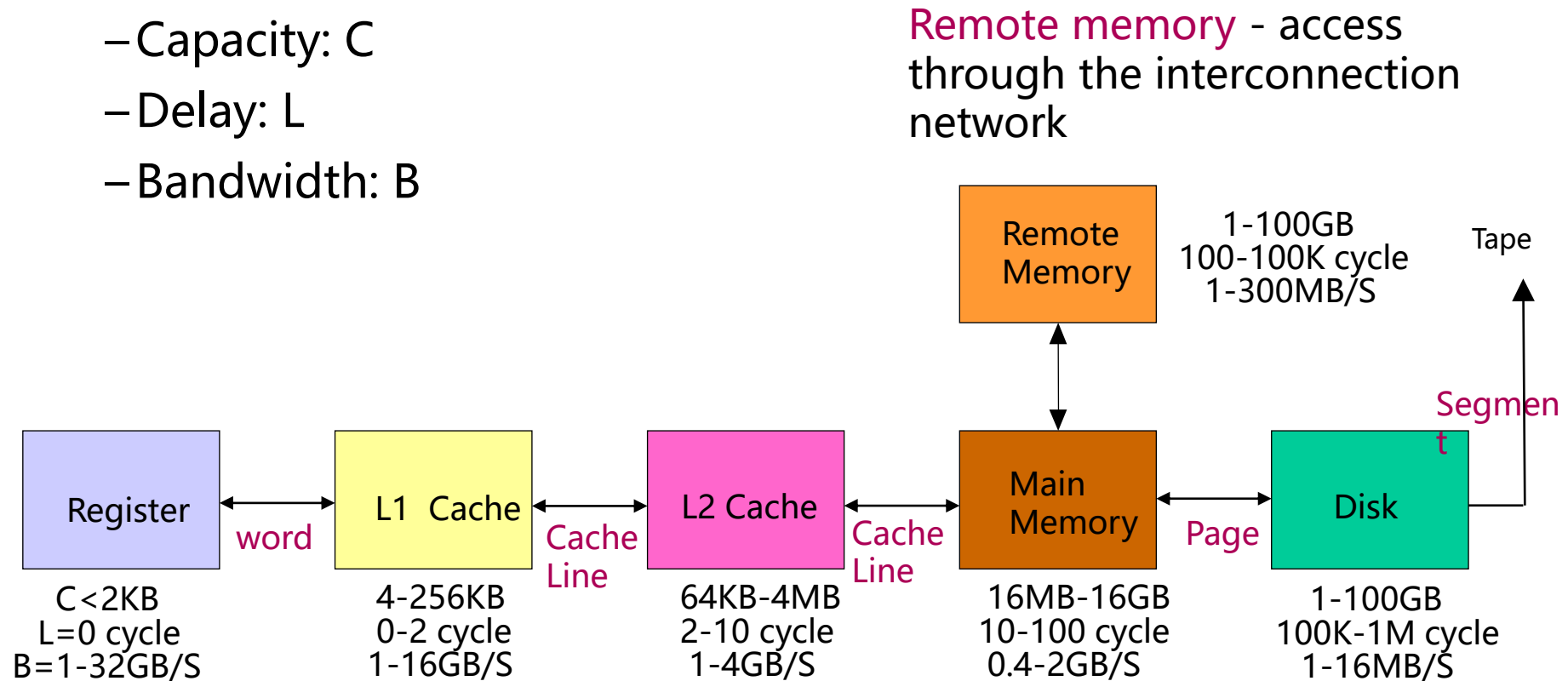
Similar Story for Storage

- 内存容量和内存访问速度差距更明显
 - 从1980-95起内存容量扩大了1000x, 每年增长50%
 - 延迟每年只降低了3% (only 2x from 1980-95)
 - 内存带宽增加了2x
- 处理器速度变快, 内存变大, 内存相对变慢
 - 需要并行传输更多地数据
 - 需要更多的cache层次



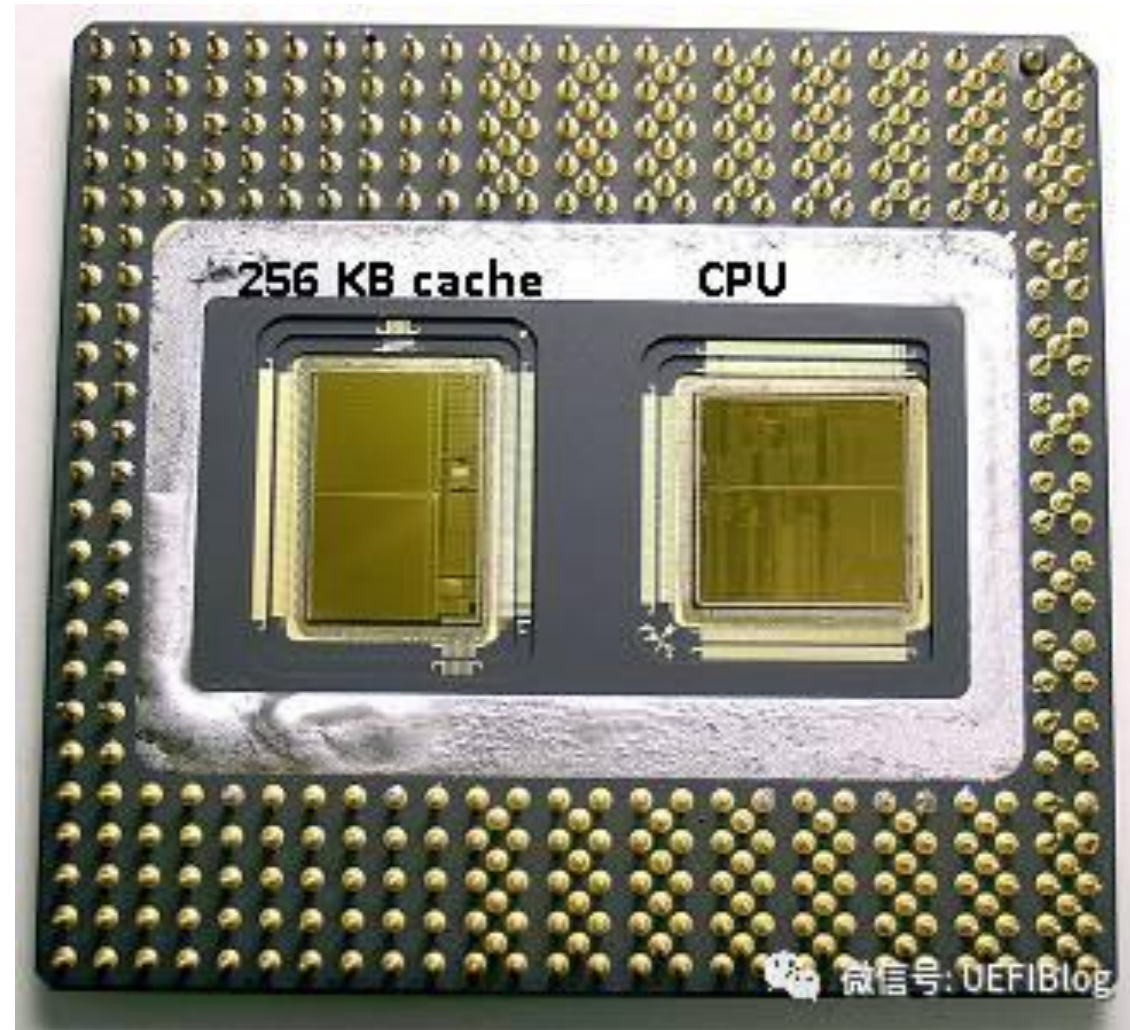
存储层次Memory hierarchy

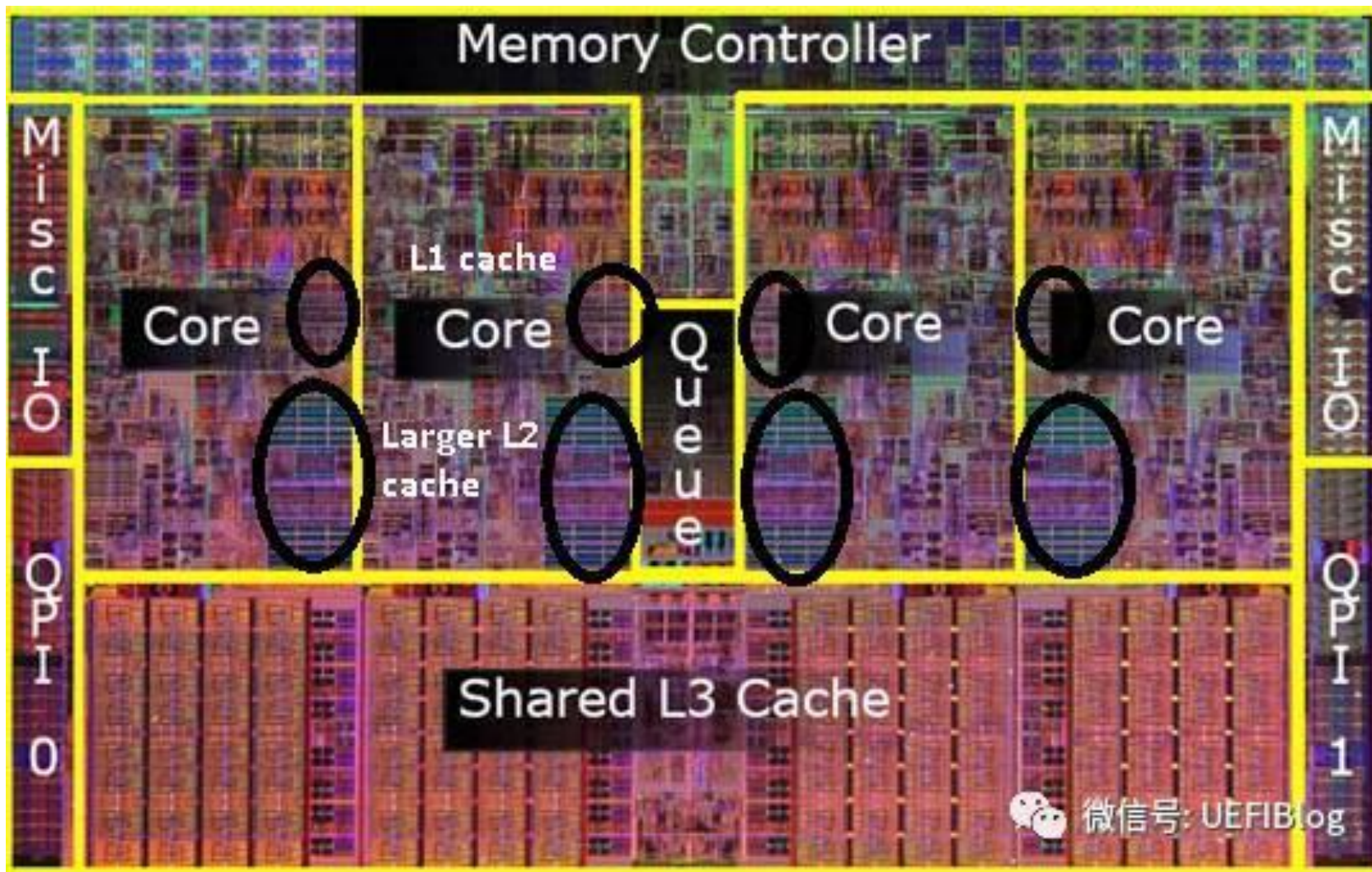
- Parameters:
 - Capacity: C
 - Delay: L
 - Bandwidth: B



Cache（高速缓冲区）

- 内存访问速度 $\ll$ 处理器执行速度
- Cache：CPU与内存间的临时存储器
 - 它的容量比内存小的多但是交换速度却比内存要快得多。
- 工作原理：
 - 当CPU要读取一个数据时，首先从缓存中查找，如果找到就立即读取并送给CPU处理；
 - 如果没有找到，就用相对慢的速度从内存中读取并送给CPU处理，同时把这个数据所在的数据块调入缓存中。





微信号: UEFIBlog



Cache类别

- L2 cache: 在结点内部的cache
- L1 cache: 在处理器内部的cache, 更小容量
 - 它的容量比内存小的多但是交换速度却比内存要快得多。
- L1 cache 连接CPU 寄存器和L2 cache, 负责缓存L2 cache 中的数据到寄存器中。

多级存储结构示意图

- Intel i7-6700 (Skylake), 4.0 GHz (Turbo Boost), 14 nm. RAM: 16 GB, dual DDR4-2400 CL15 (PC-19200).
 - L1 Data cache = 32 KB, 64 B/line, 8-WAY.
 - L1 Instruction cache = 32 KB, 64 B/line, 8-WAY.
 - L2 cache = 256 KB, 64 B/line, 4-WAY
 - L3 cache = 8 MB, 64 B/line, 16-WAY
 - L1 Data Cache Latency = 4 cycles for simple access via pointer
 - L1 Data Cache Latency = 5 cycles for access with complex address calculation (size_t n, *p; n = p[n]).
 - L2 Cache Latency = 12 cycles
 - L3 Cache Latency = 42 cycles (core 0) (i7-6700 Skylake 4.0 GHz)
 - L3 Cache Latency = 38 cycles (i7-7700K 4 GHz, Kaby Lake)
 - RAM Latency = 42 cycles + 51 ns (i7-6700 Skylake)

解决方案

- 为了编制发挥处理器峰值性能的高性能并行程序
- 一个是cache 的映射策略，即cache 是如何从内存中取数并存储的；
- 另一个是结点内部或者结点之间内存的访问模式。

Cache使用原理

- Cache以cache线(line) 为其基本组成单位，每条cache线包含L个字，每个字为8个字节。
 - 例如， $L = 4$ ，则表示cache线包含 $4 \times 8 = 32$ 个字节；
- 内存空间分割成块（block），每块大小与cache 线长度一致（L个字）。
- 数据在内存和cache 之间的移动，以cache 线为基本单位：
- 数据从内存调入cache时，不是以该单个数据字为单位，而是以该数据所在的内存块为单位，将该块的L个字一次调入cache，存储在对应的cache线中。
- 如果cache 中的数据单元要求写入内存空间，则也必须以cache 线为单位，即该数据单元所在cache 线中的所有内容一次写入内存中对应的块中。

Cache的作用

```
for(int i=0; i < M; i++)  
{  
    a(i) = a(i) + 5.0 * b(i);  
}
```

如果没有cache，则内存读访问次数为 $2M$ 次。

如果有cache，则内存访问次数下降为 $2M/L$ 次。

原因：程序的数据访问具有局部性，即程序中连续使用的数据一般存储在内存的连续位置。因此，通过cache 线的一次调入，随后的数据访问可能就落在cache 线中，从而达到减少内存访问的次数。

Cache设计的几个关键问题

- cache 容量 价格和性能的平衡
- cache 线大小 合适的长度
 - cache 线越大，则一次载入的内存数据也越多，提高性能的潜力就越大。但是，给定 cache 的容量，则cache 线越大，cache 线的条数就越少，产生cache 访问冲突的可能性就越大。cache 线一般为4-8 个
- cache 个数 两级到三级
- cache 的映射策略
 - 按映射策略，内存块的数据能够且只能复制到被映射的cache 线中，而cache 线中的数据也能够且只能被映射到对应的内存块中。
 - 可分为直接映射策略， K-路组关联映射策略和全关联映射策略（理论）三种。

Cache设计的几个关键问题 (2)

- cache 线的置换策略

- 对K-路组关联映射策略，当某个内存块请求被置入时，如何选择组中的某条cache 线，将其数据写回内存（如果该条cache 线的数据被修改）。
- LRU (Least Recently Used) 算法：置换没引用时间最长的cache线；
- FIFO (First Input First Output) 算法：置换最先置入的cache线；
- LFU (Least Frequently Used) 算法：置换使用频率最低的cache线；
- 随机算法：随机选择一条cache 线置换。

- cache 数据一致性策略

- 保持cache 数据和内存数据的一致性。
- Write-through 策略：cache 线中的数据一旦被修改，则立即写入内存块。它的缺点是，增加了许多不必要的内存访问。
- Write-back 策略：当且仅当要求进行cache 线置换时，或者有外部请求访问内存块时，将cache 线的数据写入内存。

目录

Flynn分类

内存访问模型

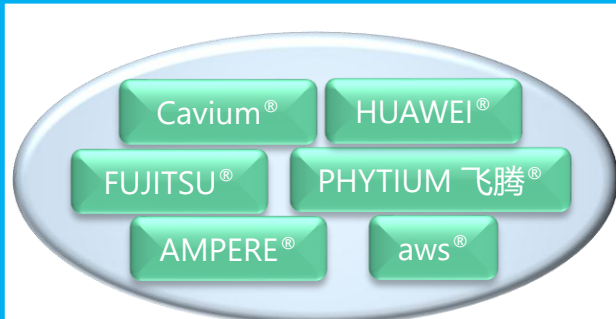
内存体系结构

处理器、协处理器与异构计算

更多系统案例

- 传统的单核处理器对计算机系统性能提升产生瓶颈
 - 必然导致多核架构的普及并且逐步成为主流
- 多核处理器
 - 通过集成多个核来提高性能，每个核的主频可以适当降低，这样可以减少功耗
 - 多核处理器中的内核采用了小型的处理器，功能相对简单，这使得多核处理器的设计和验证的周期比较短，开发风险和成本能够有效地降低，而且便于优化和重新设计。
 - 在多核处理器中，如果一个程序采用了线程级并行编程，那么这个程序在运行时可以把并行的线程同时交付给两个核心分别处理，因而程序运行速度得到极大提高。这些程序往往可以不作任何改动就直接运行在双核电脑上。多核处理器与传统的对称多处理器系统相近，所以线程级的应用可以比较容易地从传统的单处理器或对称多处理器平台移植到多核环境中。
 - 当在多核处理器上同时运行多个单线程程序的时候，操作系统会把多个程序的指令分别发送给多个内核，从而使得同时完成多个程序的速度大大加快

主流计算架构差异化特点

	 X86	 ARM	 POWER
指令集	• CISC, 复杂指令集	• RISC, 精简指令集	• RISC, 精简指令集
架构	• 重核架构, 高性能高功耗	• 多核架构, 均衡的性能功耗比	• 重核架构, 高性能内核
工艺及技术	• 14 nm, 摩尔定律放缓	• 7 nm, 业界领先的制程工艺	• 14 nm
生态	• 生态非常成熟, 通用性强	• 生态正在快速发展与完备	• 生态局限, 聚焦大小型机和HPC
开放性	• 封闭架构, 英特尔及AMD主导	• 开放平台, IP授权的商业模式	• 开放平台 (2019.8), IBM主导

ARM处理器的起源

- 1978年12月5日，物理学家赫尔曼·豪泽和工程师Chris Curry，在英国剑桥创办了CPU公司（Cambridge Processing Unit），主要业务是为当地市场供应电子设备。1979年，CPU公司改名为Acorn计算机公司。
- 起初，Acorn公司打算使用摩托罗拉公司的16位芯片，但是发现这种芯片太慢也太贵。他们转而向Intel公司索要80286芯片的设计资料，但是遭到拒绝，于是被迫自行研发。
- 1985年，Roger Wilson和Steve Furber设计了他们自己的第一代32位、6M Hz的处理器，用它做出了一台RISC指令集的计算机，简称ARM（Acorn RISC Machine）。这就是ARM这个名字的由来。
- RISC的全称是"精简指令集计算机"（Reduced Instruction Set Computer），它支持的指令比较简单，所以功耗小、价格便宜，特别适合移动设备。早期使用ARM芯片的典型设备，就是苹果公司的牛顿PDA。

ARM处理器的特点

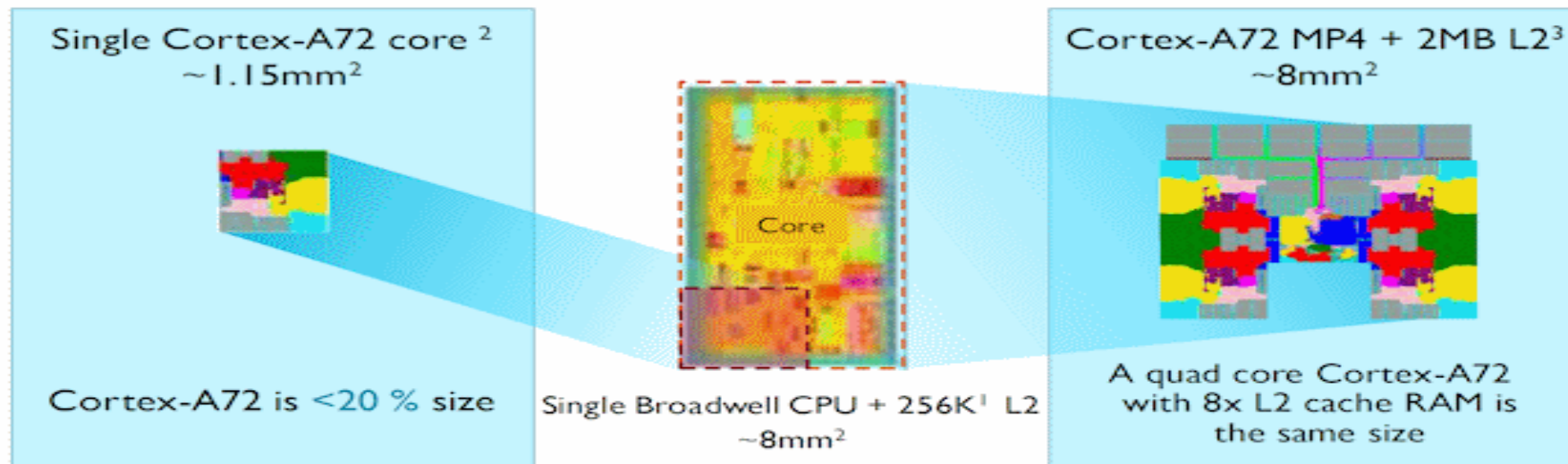
- ARM处理器的特点：耗电少功能强、16/32/64位指令集、合作伙伴众多
 - 体积小、低功耗、低成本、高性能；
 - 支持Thumb（16位）/ARM（32/64位）指令集，能很好地兼容8位/16位器件；
 - 大量使用寄存器，指令执行速度更快；
 - 大多数数据操作都在寄存器中完成；
 - 寻址方式灵活简单，执行效率高；
 - 指令长度固定。

ARM架构优势

- ARM的众核横向扩展空间优势明显

一个ARM核的面积仅为X86核的1/7,
同样的芯片尺寸下, ARM的核数是X86的4倍以上。

Cortex-A72: Ideal for dense compute environments



单个ARM核的面积大约为1.15mm²

单个X86核的面积大约为8mm²

支持高能耗比

- ARM具有与X86架构不可对比的优势，该优势就是：功耗
- 通过测试，ARM架构处理器能耗平均比x86架构处理器低20%~25%

Power Consumption Comparison				
	X86 (Totally)	ARM (Totally)	X86 (CPU)	ARM (CPU)
Idle	3.182W	2.474W	70.2mw	36.4mw
Cold boot	5.358W	3.280W	800mW	216mW
Sunspider0.9.1	4.775W	3.704W	722mW	520mW
Kraken	4.738W	3.582W	829mW	564mW
RIABench	3.962W	3.294W	379mW	261mW
WebXPRT	4.617W	3.225W	663mW	412mW
TouchXPRT (Photo Enhance)	4.789W	3.793W	913mW	378mW
GPU Workload	5.395W	3.656W	1432mW	488mW

ARM HPC: SIMD 发展历程

- NEON—从2004 年的ARMv7开始引入：
 - ARMv7: 仅整数定点及non-IEEE的SP, 16x128b regs;
 - AArch64: 完整支持IEEE-SP/DP, 64b int操作, 32x128b regs;
 - 近十几年变化并不大: 专注移动市场light-weight multimedia应用;
 - 指令扩展, 特性支持, 向量长度等都比较缓慢
- SVE (Scalable vector extension) introduced in 2016 with Fujitsu
 - Target on HPC&ML with rich vector instructions and wider vector;
 - Better scalability across multiple implementations;
 - NOT ext. of NEON, totally new!
- SVE2: 将会出现在下一代架构授权中
 - 支持NEON-style fixed-point DSP指令: 以VLA的方式;
 - 指令支持加速更多应用: ML, CV, LTE, in-memory database etc;
 - 优化auto-vectorization。

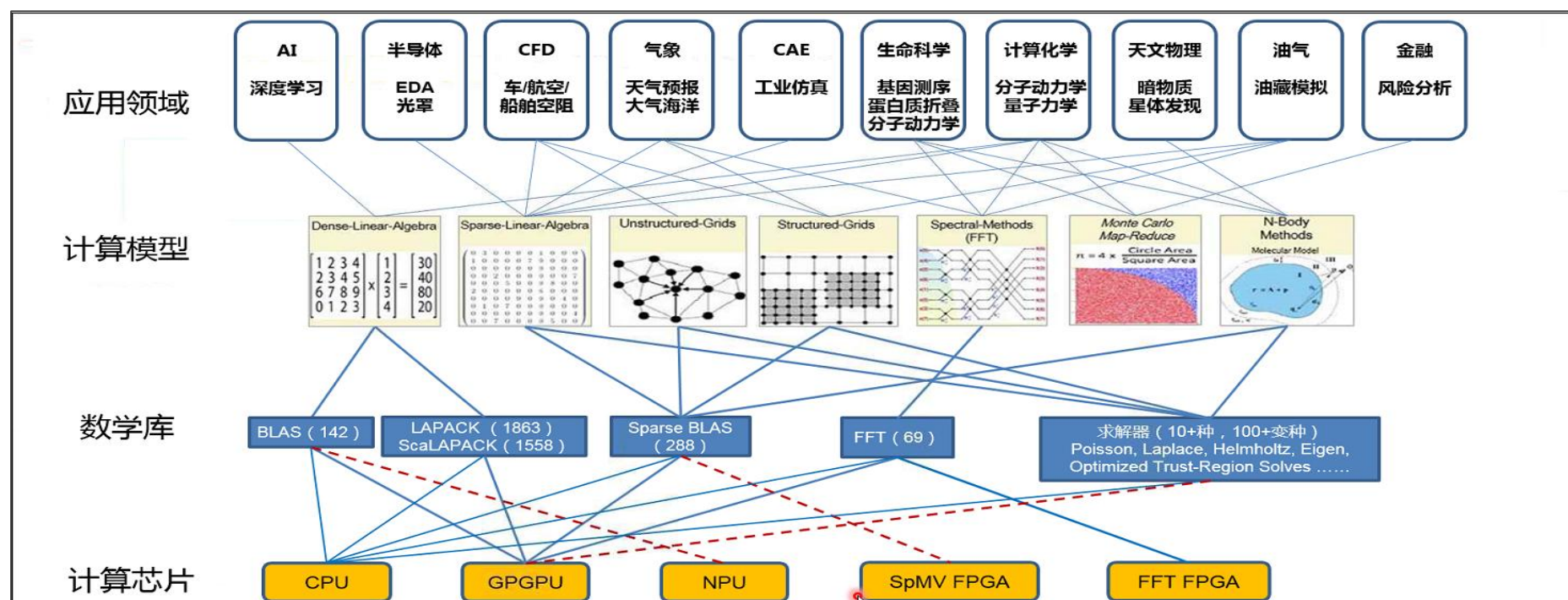
ARM HPC: 跟主流CPU FP应当算力挑战

- SVE是ARM的关键特性!
 - 显著提升在通用计算尤其HPC领域的竞争力
 - 但是也面临着如何用好SVE以及构建相应生态的挑战

	#core	frequency (no turbo)	SIMD VL	SIMD unit cnt./core	TFLOPS (DP)
AMD zen3	64	2.25	256	3	~3.4/2.4
Intel V6 9282 (cascadelake-AP)	56	2.6	512	2	~3.0
A64FX (post-K)	48	1.8	512	2	~2.7
Intel V6 8280	28	2.7/1.8	512	2	~1.6
AMD 7702	64	2.25	256	2	~1.6
Intel V5 8180	28	2.5/1.7	512	2	~1.5
Intel V5 6148	20	2.4/1.6	512	2	~1.0
Hi1620	64	2.6	128	2	~0.6
Hi1616	32	2.4	128	1	~0.3

ARM HPC: 通过SVE、算子、算法给HPC赋能

- 通过计算能力的提升及主要算子库的优化，达到算力，能效的提升和领先
 - 应用领域分析识别：算力需求+计算模型+数学库依赖
 - 应用的计算效率优化：算法优化+算力分布+算子调优



更强的计算能力

高性能

930+

SPECint®_rate_base2006 评估跑分

高吞吐

内存带宽: **2.4x ↑**

I/O 总带宽: **1.7x ↑**

网络带宽: **10x ↑**

高集成

1 颗 = **4** 颗芯片

(CPU, 南桥、网卡、SAS控制器)

高效能

35% ↑

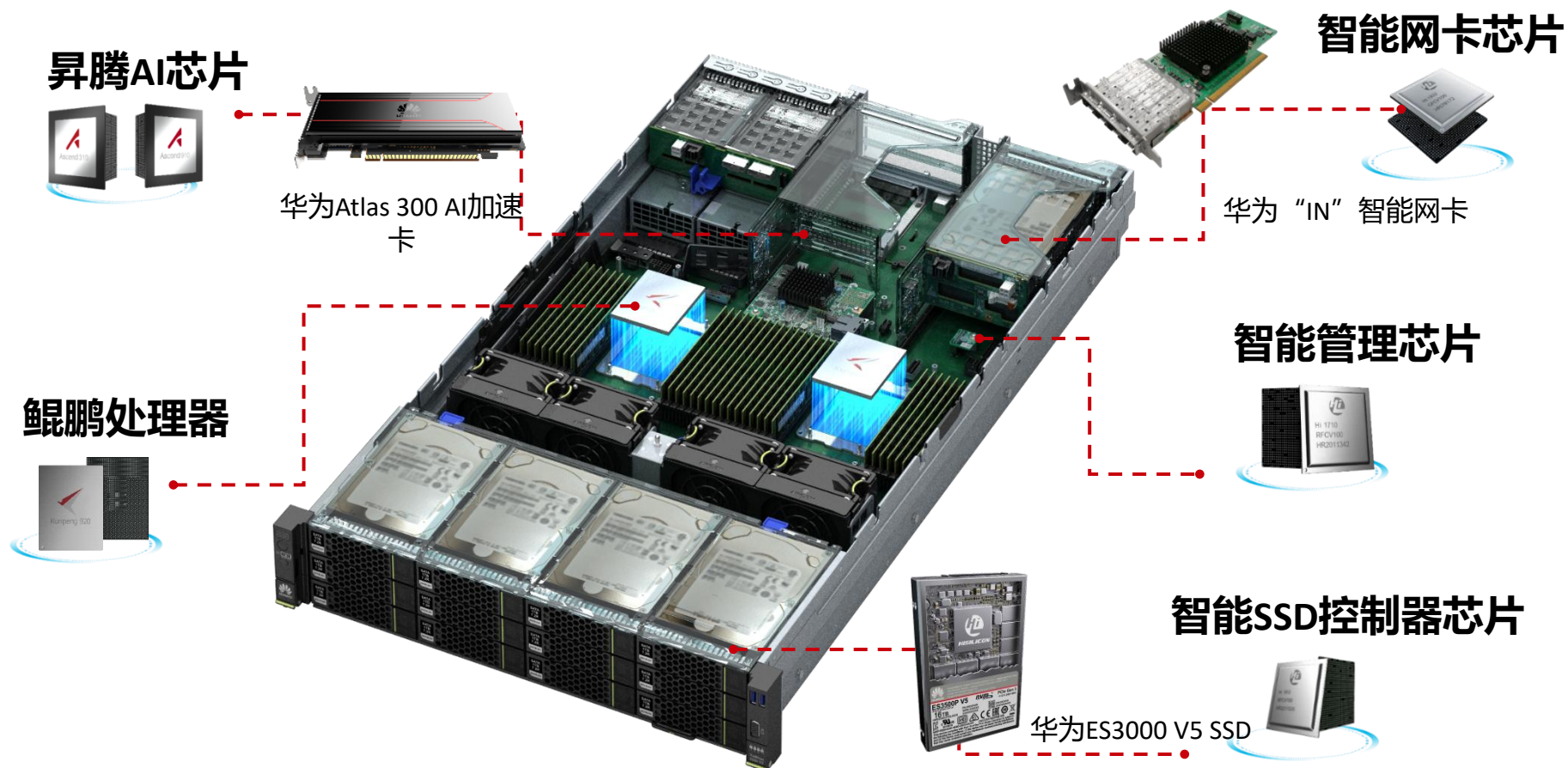
*基于鲲鹏920-6426 vs 鲲鹏916处理器的华为实验室测试对比数据, 结果在不同环境中可能有偏差



工艺: 7nm | 多核: 64核 | 内存: 8通道 | 接口: PCIe 4.0 & 100GE

业界最高性能ARM-Based处理器

计算平台——华为TaiShan服务器 (1)



AI芯片发展历史

- 人工智能的四大要素：数据、算法、场景、**算力**
- **AI芯片**：也被称为AI加速器，即专门用于处理人工智能应用中的大量计算任务的功能模块。
- AI芯片的发展经历了从CPU到GPU，再到FPGA和ASIC芯片的发展历程。

AI芯片现状 — CPU

• CPU(Central Processing Unit)

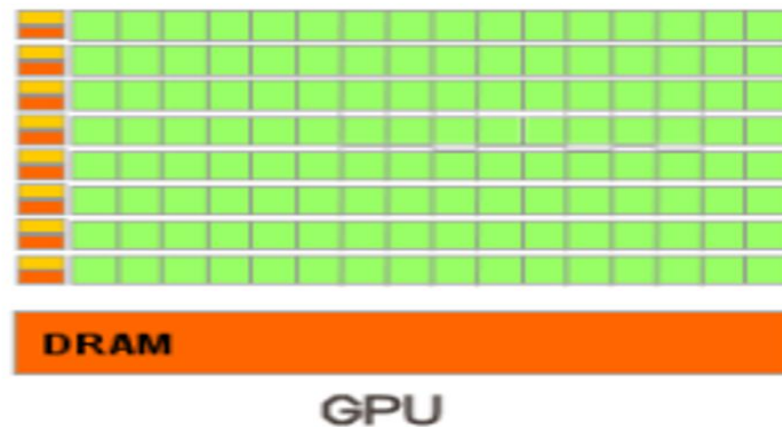
- 早期计算机性能随着摩尔定律逐年稳步提升。
- 增加核数提升性能，多核处理器更好的满足了软件对硬件的速度需求，但带来功耗和成本增加。
- 增加指令（修改架构）的方式提升AI性能
 - Intel（CISC架构）加入AVX512等指令，在ALU计算模块加入矢量运算模块（FMA）
 - ARM（RISC架构）加入Cortex A等指令集并计划持续升级
- 提高频率提升性能，但提升空间有限，同时高主频会导致芯片出现功耗过大和过热问题。

• GPU(Graph Processing Unit)

- GPU(Graph Processing Unit)在矩阵计算和并行计算上具有突出的性能，异构计算的主力，最早作为深度学习的加速芯片被引入AI领域，且生态成熟。
- Nvidia沿用GPU架构，对深度学习主要向两个方向发力：
 - **丰富生态**：推出cuDNN针对神经网络的优化库，提升易用性并优化GPU底层架构
 - **提升定制性**：增加多数据类型支持（不再坚持float32，增加int8等）；添加深度学习专用模块（如引入并配备张量核的改进型架构，V100的TensorCore）
- 当前主要问题在于：**成本高，能耗比低，延迟高。**

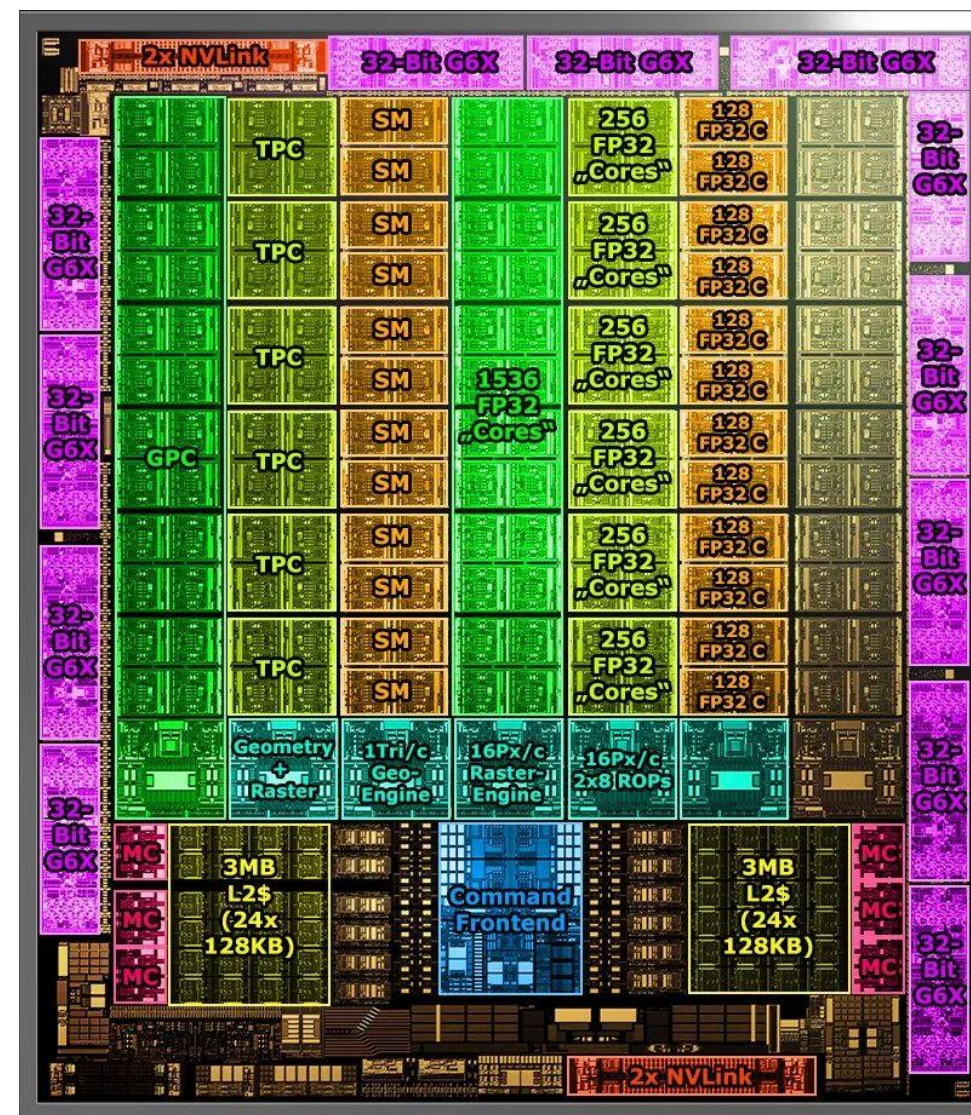
GPU、CPU设计比对

- **GPU主要面对类型高度统一、相互无依赖的大规模数据和不需打断的纯净计算环境。**
 - 拥有若干由数以千计的更小的核心（专为同时处理多重任务而设计）组成的大规模并行计算架构
 - 基于大吞吐量设计
 - 有很多ALU和很少cache（和CPU目的不同，为thread提供服务），缓存合并访问DRAM，带来时延问题。
 - 控制单元合并访问。
 - 大量ALU实现大量thread并行掩盖时延问题。
 - 擅长计算密集和易于并行的程序



Nvidia GA102 Ampere

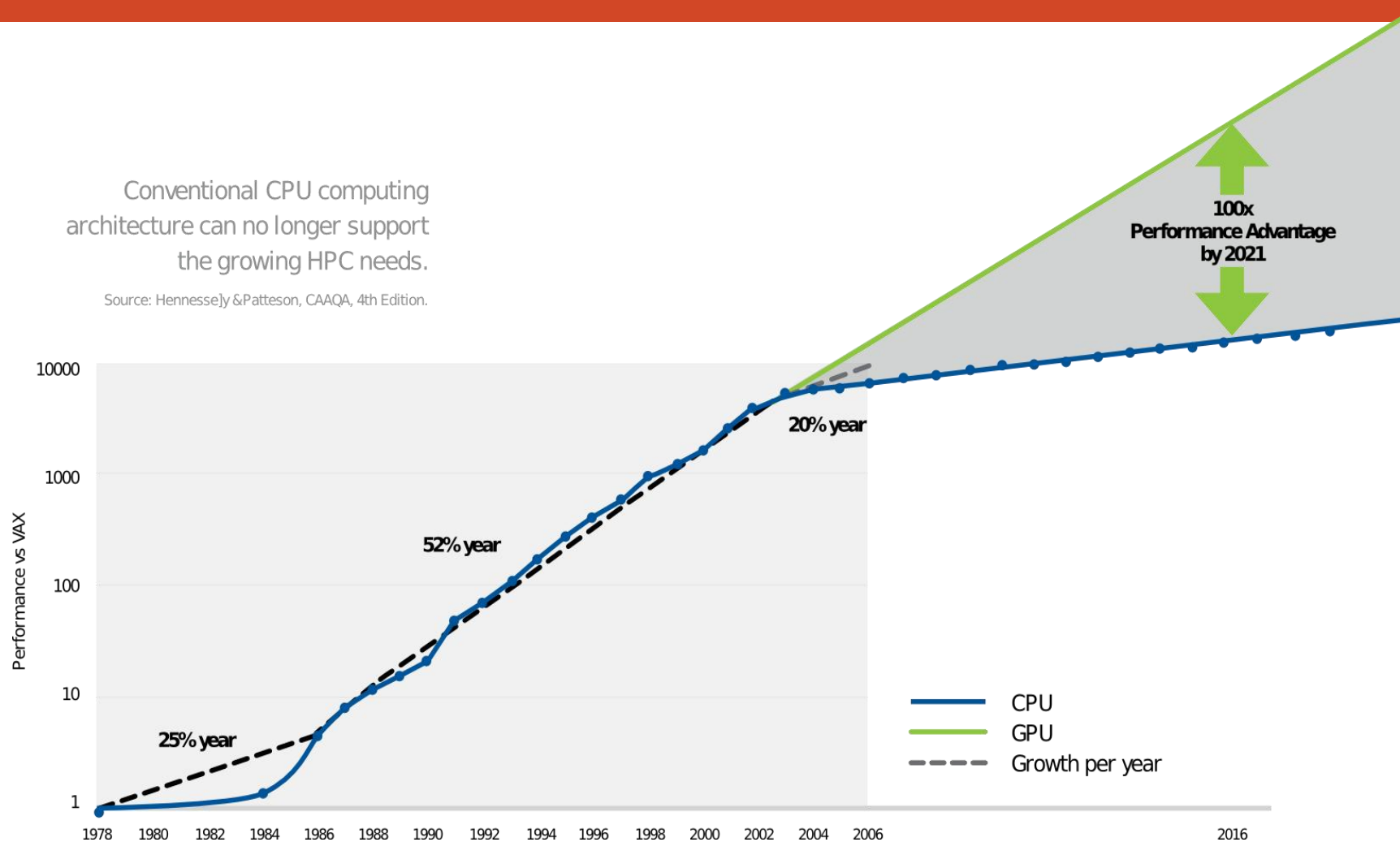
- 每个Graphics Processing Clusters (GPC) 有12个 Streaming Multiprocessors (SM), 共84个SM
- 每个SM有128 CUDA核, 4 Tensor 核, 1 RT核
- 共10,752 CUDA 核, 336 Tensor核, 84 RT核



GA10x Streaming Multiprocessor (SM)



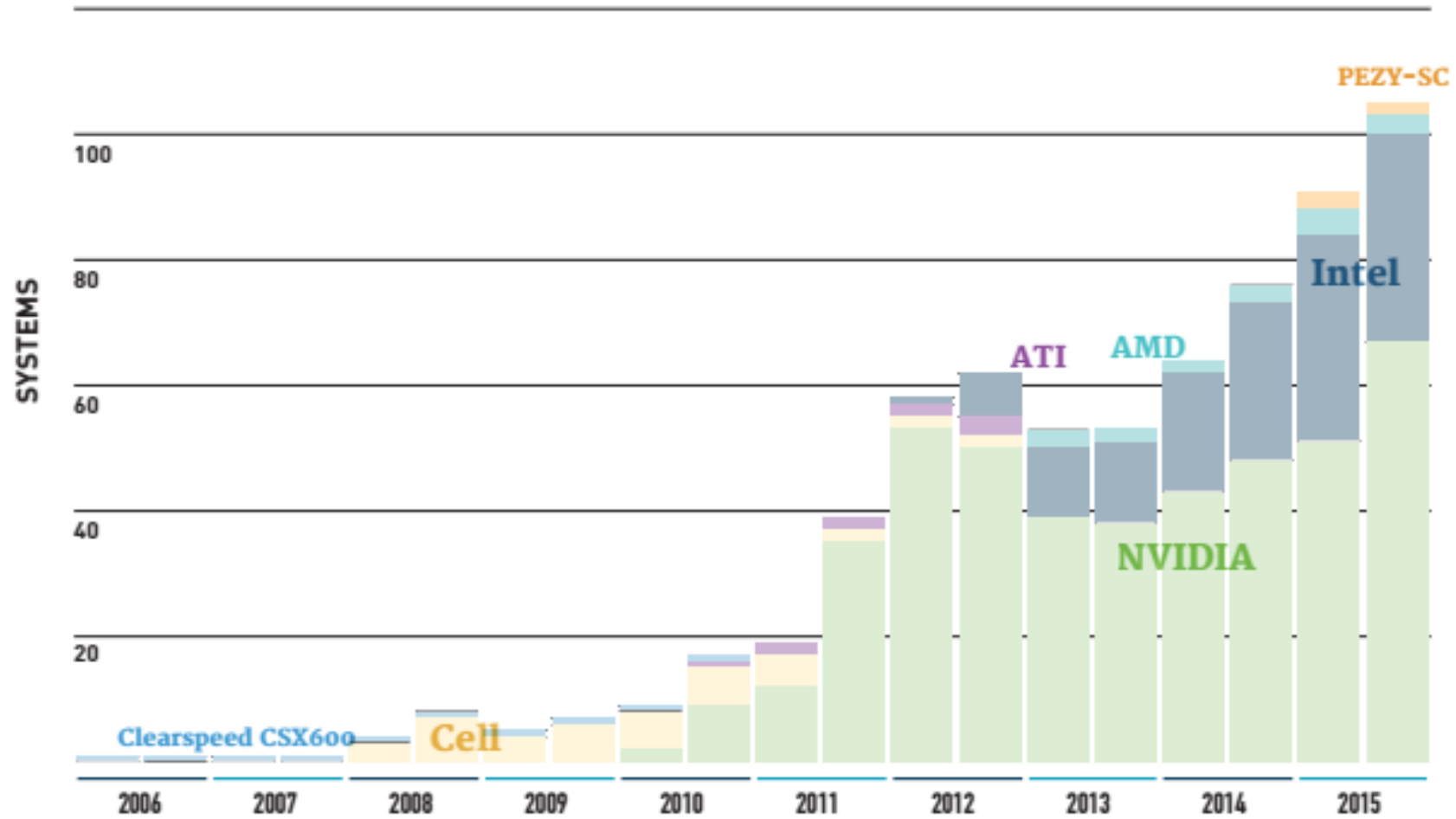
The Gap Between CPU and GPU



ref: [Tesla GPU Computing Brochure](#)

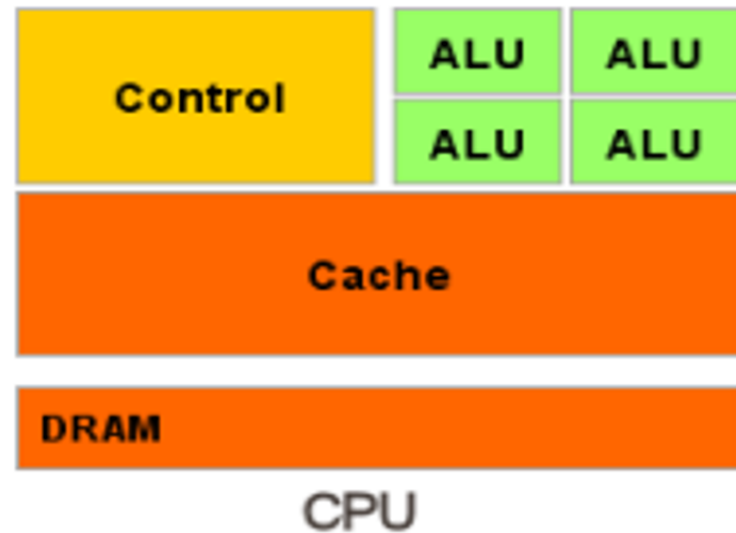


ACCELERATORS/CO-PROCESSORS



GPU、CPU设计比对

- CPU需要很强通用性处理不同数据类型，同时需要逻辑判断，还会引入大量分支跳转和中断处理。
 - 由专为串行处理而优化的几个核心组成
 - 基于低延时设计
 - 强大的ALU单元，可在很短时钟周期完成计算
 - 大量缓存降低延时
 - 高时钟频率
 - 复杂逻辑控制单元，多分支程序可通过分支预测能力降低时延
 - 对于依赖之前指令结果的部分指令，逻辑单元决定指令在pipeline中的位置实现数据快速转发
 - 擅长逻辑控制、串行运算



AI芯片现状 — FPGA

• FPGA

- 采用HDL可编程方式，灵活性高，可重构（烧），可深度定制。
- 可通过多片FPGA联合将DNN模型加载到片上进行低延迟计算，计算性能优于GPU，但由于需考虑不断擦写，性能达不到最优（冗余晶体管和连线，相同功能逻辑电路占芯片面积更大）。
- 由于可重构，供货风险和研发风险较低，成本取决于购买数量，相对自由。
- 设计、流片过程解耦，**开发周期较长**（通常半年），**门槛高**。

AI芯片现状 — TPU

- TPU(Tensor Processing Unit)

- 谷歌从2006年起致力于将专用集成电路ASIC的设计理念应用到神经网络领域，发布了支撑深度学习开源框架TensorFlow的人工智能定制芯片TPU(Tensor Processing Unit)。
- 利用大规模脉动阵列结合大容量片上存储来高效加速深度神经网络中最为常见的卷积运算：
 - 脉动阵列可用来优化矩阵乘法和卷积运算，以达到提供更高算力和更低能耗的作用。



AI芯片应用场景（1）

- 从技术架构来看，大致分为四个类型：

- CPU（Central Processing Unit，中央处理器）：是一块超大规模的集成电路，是一台计算机的运算核心（Core）和控制核心（Control Unit）。它的功能主要是解释计算机指令以及处理计算机软件中的数据。
- GPU（Graphics Processing Unit，图形处理器）：又称显示核心、视觉处理器、显示芯片，是一种专门在个人电脑、工作站、游戏机和一些移动设备（如平板电脑、智能手机等）上图像运算工作的微处理器。
- ASIC（Application Specific Integrated Circuit，专用集成电路）：适合于某一单一用途的集成电路产品。
- FPGA（Field Programmable Gate Array，现场可编程门阵列）：其设计初衷是为了实现半定制芯片的功能，即硬件结构可根据需要实时配置灵活改变。

AI芯片应用场景（2）

- 从业务应用来看，可以分为Training（训练）和Inference（推理）两个类型：
- Training环节通常需要通过大量的数据输入，或采取增强学习等非监督学习方法，训练出一个复杂的深度神经网络模型。训练过程涉及海量的训练数据和复杂的深度神经网络结构，运算量巨大，需要庞大的计算规模，对于处理器的计算能力、精度、可扩展性等性能要求很高。常用的例如NVIDIA的GPU集群、Google的TPU等。
- Inference环节指利用训练好的模型，使用新的数据去“推理”出各种结论。虽然Inference的计算量相比Training少很多，但仍然涉及大量的矩阵运算。在推理环节，GPU、FPGA和ASIC都有很多应用价值。

人工智能芯片行业背景

- 基于FPGA的半定制人工智能芯片，利用具备可重构性的FPGA芯片来半定制的人工智能芯片是最佳选择。
- 针对深度学习算法的全定制人工智能芯片，该类芯片是完全采用ASIC设计方法全定制，性能、功耗和面积等指标面向深度学习算法都做到了最优。谷歌的TPU芯片、我国中科院计算所的寒武纪深度学习处理器芯片就是这类芯片的典型代表。
- 类脑计算芯片，这类芯片的设计目的不再局限于仅仅加速深度学习算法，而是在芯片基本结构甚至器件层面上希望能够开发新的计算机体系结构。这类芯片的典型代表是IBM的Truenorth芯片。

昇腾 (Ascend) AI处理器

- **NPU (Neural-Network Processing Units, 神经网络处理器)**：在电路层模拟人类神经元和突触，并且用深度学习指令集直接处理大规模的神经元和突触，一条指令完成一组神经元的处理。
- NPU的典型代表 —— **华为昇腾AI芯片 (Ascend)、IBM的TrueNorth。**



Ascend 310 ——
面向推理场景

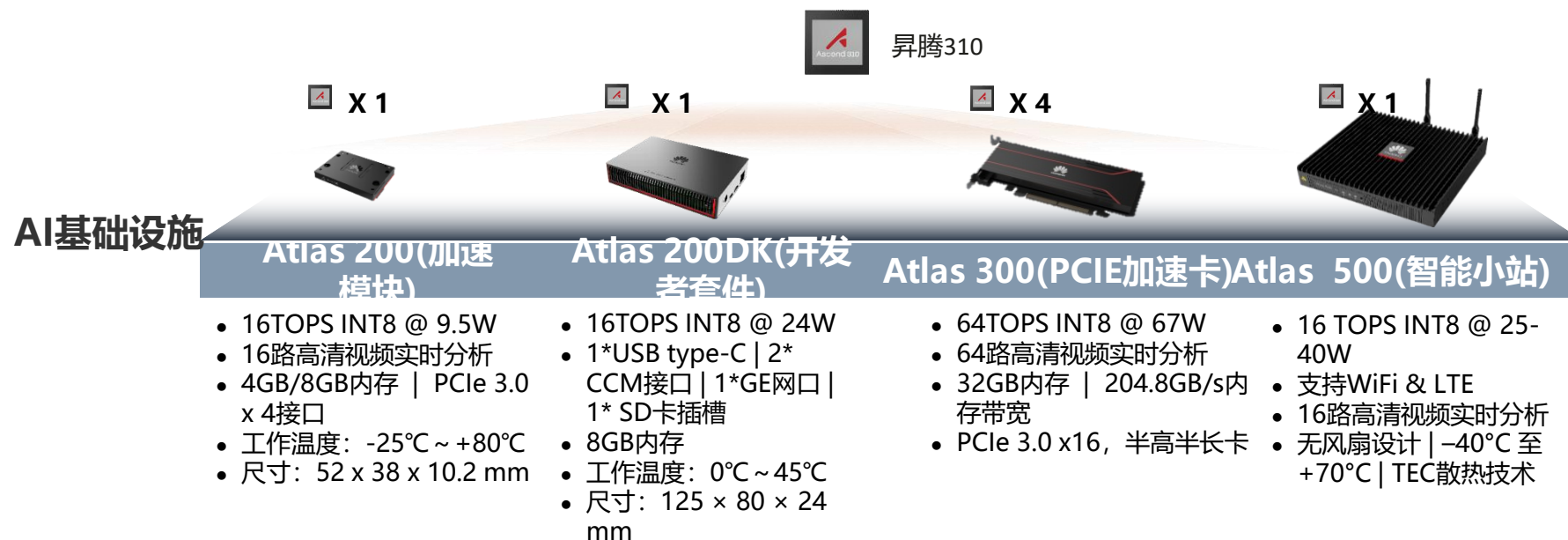
- Ascend 310
- 架构: 达芬奇
- 半精度 (FP16): 8 Tera-FLOPS
- 整数精度 (INT8) : 16 Tera-OPS
- 16 通道 全高清 视频解码器 – H.264/265;
- 1 通道 全高清 视频编码器 – H.264/265;
- **最大功耗: 8W**
- 12nm FFC



Ascend 910 ——
面向训练场景

- Ascend 910
- 架构: 达芬奇
- 半精度 (FP16): 256 Tera-FLOPS
- 整数精度 (INT8) : 512 Tera-OPS
- 128 通道 全高清 视频解码器 – H.264/265;
- 最大功耗: 350W
- 7nm

基于昇腾AI处理器的产品形态



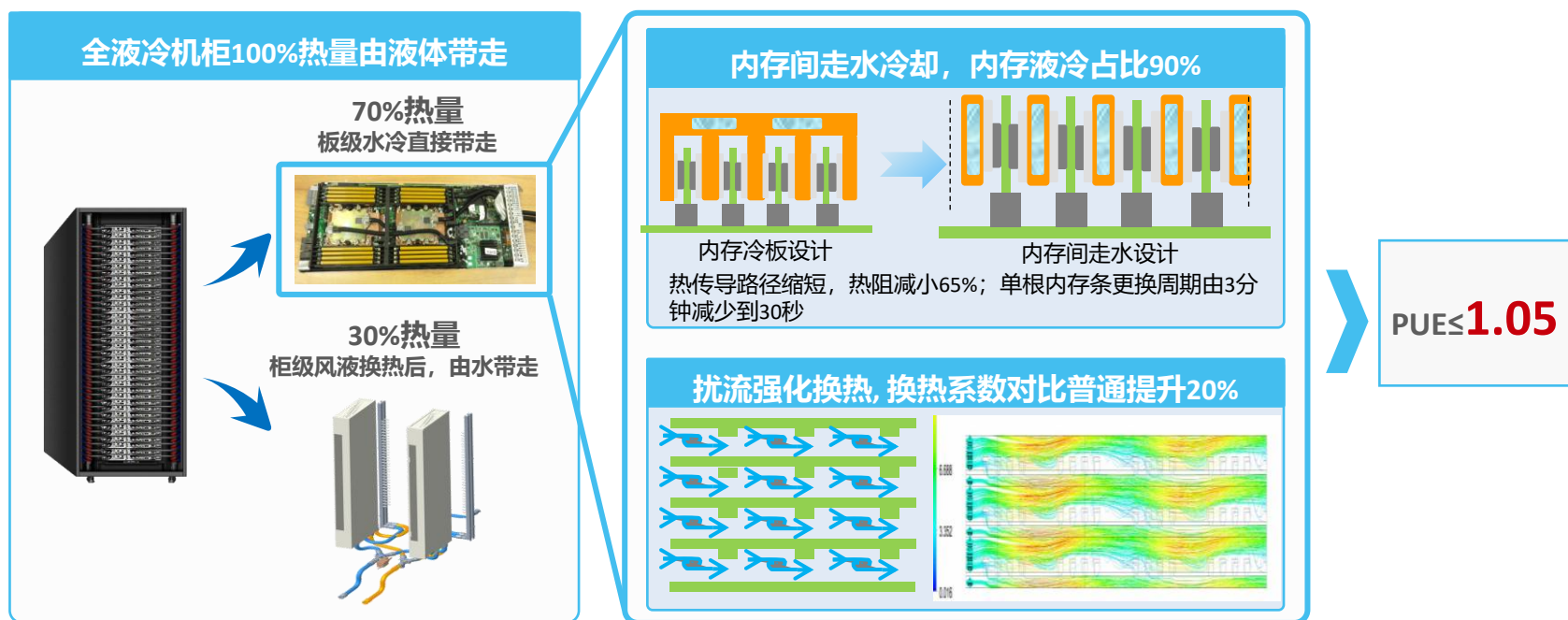
基于昇腾AI处理器的产品形态

AI解决方案



基础设施——更高的能效

高效液冷解决方案，PUE低至1.05



基础设施——能效参数简介

- PUE (Power Usage Effectiveness) : 电力使用效率
 - **PUE = 数据中心总能耗/IT设备能耗**
 - PUE越接近1表明非IT设备耗能越少, 即能效水平越高
- 当前, 国外先进的数据中心机房PUE值通常小于2, 而我国的大多数数据中心的PUE值在2-3之间
- 数据中心的冷却占总功耗的40%左右, 机房冷却主要由空调负责, 所以降低空调的耗电量可以有效的降低PUE的值
- 要做到最佳节能, 降低服务器等IT设备的功耗, 才是最有效的方法

谢谢